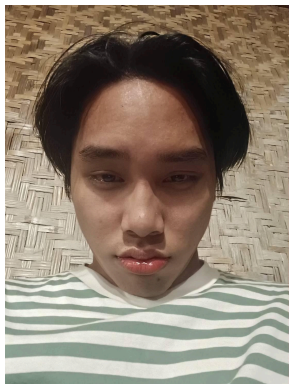
TUGAS BESAR

IF2224 - Teori Bahasa Formal dan Otomata

Milestone 2

Syntax Analysis




Benedictus Nelson - 13523150

Rainaldi Pratama F. Sembiring - 13524117

Jonathan Alveraldo Bangun - 13524120

Ramadhian Nabil Firdaus Gumay - 13524126

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

DAFTAR ISI

A. TEORI SINGKAT

Syntax Analysis merupakan salah satu tahap utama dalam proses kompilasi, yang berada setelah lexical analysis. Jika lexical analysis bertugas memecah kode sumber menjadi token-token, maka syntax analysis bertugas memeriksa apakah token-token tersebut tersusun dengan benar sesuai dengan grammar atau aturan tata bahasa dari bahasa pemrograman yang digunakan. Dengan kata lain, tahap ini memastikan bahwa rangkaian token yang dihasilkan lexer benar-benar membentuk struktur sintaks yang valid.

Peran syntax analysis sangat penting karena pada tahap inilah compiler mulai memahami bentuk program secara struktural. Parser tidak hanya membaca token satu per satu, tetapi juga memeriksa hubungan antar token berdasarkan aturan produksi dalam grammar. Apabila terdapat urutan token yang tidak sesuai dengan aturan tersebut, parser akan mendeteksi adanya syntax error dan melaporkannya kepada pengguna. Hal ini membantu memastikan bahwa program yang diproses oleh compiler memiliki bentuk yang benar sebelum dilanjutkan ke tahap semantic analysis dan code generation.

Dalam implementasi milestone ini, syntax analysis dilakukan menggunakan metode Recursive Descent Parser. Metode ini merupakan teknik top-down parsing yang bekerja dengan cara menguraikan grammar menjadi kumpulan fungsi rekursif. Setiap fungsi merepresentasikan satu aturan produksi dalam grammar, dan fungsi-fungsi tersebut saling memanggil untuk mencocokkan token input dengan struktur bahasa yang telah didefinisikan. Pendekatan ini cukup intuitif karena alur parsing mengikuti bentuk grammar secara langsung, sehingga lebih mudah dipahami dan diimplementasikan untuk grammar yang sudah dirancang secara jelas.

Grammar bahasa Arion pada proyek ini di-hardcode ke dalam program parser. Artinya, seluruh aturan sintaks bahasa dimasukkan secara eksplisit ke dalam implementasi, mulai dari struktur program utama, deklarasi, tipe data, prosedur, fungsi, hingga bentuk-bentuk statement seperti assignment, if, while, repeat, for, dan case. Dengan pendekatan ini, parser dapat mengenali berbagai konstruksi bahasa Arion secara terstruktur dan memastikan bahwa setiap bagian program sesuai dengan format yang diperbolehkan.

Output akhir dari syntax analysis adalah Parse Tree, yaitu representasi pohon yang menunjukkan bagaimana sebuah program dibentuk dari grammar bahasa yang digunakan. Parse Tree menggambarkan seluruh langkah derivasi secara lengkap, mulai dari simbol awal hingga terminal-terminal yang sesuai dengan token input. Setiap node pada pohon mewakili simbol terminal maupun non-terminal, sehingga struktur hubungan

antar elemen sintaks dapat terlihat secara rinci. Representasi ini sangat berguna untuk memahami bagaimana program diparse dan bagaimana aturan grammar diterapkan pada input.

Secara keseluruhan, syntax analysis berfungsi sebagai jembatan antara token hasil lexical analysis dan representasi struktur program yang dapat diproses lebih lanjut oleh compiler. Tahap ini memastikan bahwa program tidak hanya tersusun dari token-token yang valid, tetapi juga mengikuti aturan sintaks yang benar. Dengan demikian, parser menjadi komponen penting dalam compiler karena berperan menjaga ketepatan struktur bahasa sebelum program masuk ke tahap analisis semantik dan proses kompilasi selanjutnya.

B. ATURAN GRAMMAR

Berikut adalah keseluruhan aturan Grammar dari bahasa Arion yang kami hardcode ke dalam algoritma Recursive Descent Parser.

1. <program> → <program-header> + <declaration-part> + <compound-statement> + period
2. <program-header> → programsy + ident + semicolon
3. <block> → <declaration-part> + <compound-statement>
4. <declaration-part> → (<const-declaration>)* + (<type-declaration>)* + (<var-declaration>)* + (<subprogram-declaration>)*
5. <const-declaration> → constsy + (ident + eql + <constant> + semicolon)+
6. <type-declaration> → typesy + (ident + eql + <type> + semicolon)+
7. <var-declaration> → varsy + (<identifier-list> + colon + <type> + semicolon)+
8. <constant> → charcon | string | [(plus | minus)? + (ident | intcon | realcon)]
9. <type> → ident | <array-type> | <range> | <enumerated> | <record-type>
10. <array-type> → arraysy + lbrack + (<range> | ident) + rbrack + ofsy + <type>
11. <range> → <constant> + period + period + <constant>
12. <enumerated> → lparent + ident + (comma + ident)* + rparent
13. <record-type> → recordsy + <field-list> + endsy
14. <field-list> → <field-part> + (semicolon + <field-part>)*
15. <field-part> → <identifier-list> + colon + <type>
16. <identifier-list> → ident + (comma + ident)*
17. <variable> → ident + (<component-variable>)*
18. <component-variable> → (lbrack + <index-list> + rbrack) | (period + ident)
19. <index-list> → (intcon | charcon | ident) + (comma + <index-list>)*
20. <subprogram-declaration> → <procedure-declaration> | <function-declaration>
21. <procedure-declaration> → proceduresy + ident + (<formal-parameter-list>)? + semicolon + <block> + semicolon

22. <function-declaration> → functionsy + ident + (<formal-parameter-list>)? + colon + ident + semicolon + <block> + semicolon
23. <formal-parameter-list> → lparent + <parameter-group> + (semicolon + <parameter-group>)* + rparent
24. <parameter-group> → <identifier-list> + colon + (ident | <array-type>)
25. <procedure/function-call> → ident + (lparent + <parameter-list>? + rparent)?
26. <parameter-list> → <expression> + (comma + <expression>)*
27. <compound-statement> → beginsy + <statement-list> + endsy
28. <statement-list> → <statement> + (semicolon + <statement>)*
29. <statement> → (<assignment-statement> | <if-statement> | <case-statement> | <while-statement> | <repeat-statement> | <for-statement> | <procedure/function-call>)?
30. <assignment-statement> → <variable> + becomes + <expression>
31. <if-statement> → ifsy + <expression> + thensy + <statement> + (elsesy + <statement>)?
32. <case-statement> → casesy + <expression> + ofsy + <case-block> + endsy
33. <case-block> → <constant> + (comma + <constant>)* + colon + <statement> + (semicolon + <case-block>?)*
34. <while-statement> → whilesy + <expression> + dosy + <statement>
35. <repeat-statement> → repeatsy + <statement-list> + untilsy + <expression>
36. <for-statement> → forsy + ident + becomes + <expression> + (tosy | downtosy) + <expression> + dosy + <statement>
37. <expression> → <simple-expression> + (<relational-operator> + <simple-expression>)?
38. <simple-expression> → (plus | minus)? + <term> + (<additive-operator> + <term>)*
39. <term> → <factor> + (<multiplicative-operator> + <factor>)*
40. <factor> → ident | intcon | realcon | charcon | string | (lparent + <expression> + rparent) | (notsy + <factor>) | <procedure/function-call> | <variable>
41. <relational-operator> → eql | neq | gtr | geq | lss | leq
42. <additive-operator> → plus | minus | orsy
43. <multiplicative-operator> → times | rdiv | idiv | imod | andsy

C. PERANCANGAN DAN IMPLEMENTASI

1. Arsitektur Program

Alur kerja program:

source code / token file -> lexer (opsional) -> list token -> parser recursive descent -> parse tree -> output txt

Program ditulis dalam bahasa C++ (standar C++11) untuk konsistensi dengan Milestone 1 dan dikompilasi melalui Makefile yang sama. Source code parser dipisah menjadi tiga modul agar setiap kelompok aturan produksi tetap modular dan mudah ditelusuri, namun seluruh fungsi tetap berada dalam satu kelas Parser agar berbagi state token, indeks pembacaan, dan flag error secara konsisten. Pembagian modulnya adalah parser_decl.cpp untuk produksi program, deklarasi, dan tipe data; parser_stmt.cpp untuk produksi statement, blok, dan subprogram; serta parser_expr.cpp untuk helper Parser sekaligus produksi ekspresi dan struktur kontrol. File main.cpp bertanggung jawab atas dispatch input (apakah file mentah atau hasil tokenisasi) dan pencetakan parse tree.

2. Implementasi

a. Struktur Data ParseTreeNode

Parse Tree direpresentasikan oleh struct ParseTreeNode yang memiliki tiga atribut yaitu label (nama node, contoh "<expression>" atau "ident"), value (nilai untuk leaf bertipe ident/intcon/realcon/charcon/string), dan children (vector pointer ke sub-pohon yang urut sesuai grammar). Struktur ini sengaja dibuat ringkas agar pencetakan parse tree dapat dilakukan dengan satu fungsi rekursif. Destruktor merekursifkan delete ke seluruh anak sehingga tidak terjadi kebocoran memori.

```
12  struct ParseTreeNode {
13      string label;
14      string value;
15      vector<ParseTreeNode*> children;
16      ParseTreeNode(string l, string v = "") : label(l), value(v) {}
17      ~ParseTreeNode() {
18          for (auto child : children) {
19              delete child;
20          }
21      }
22  };
```

b. State Internal Kelas Parser

Kelas Parser menyimpan tiga state internal yaitu vector<Token> tokens (seluruh token yang diterima dari Lexer), int currentTokenIndex (posisi pembacaan saat ini, analog pita mesin), dan bool hasError (penanda bahwa parsing sudah gagal

untuk mencegah kaskade error). Konstruktor menginisialisasi indeks ke nol dan flag error ke false:

```
4   Parser::Parser(const vector<Token>& tokens) {
5     this->tokens = tokens;
6     this->currentTokenIndex = 0;
7     this->hasError = false;
8   }
```

c. Fungsi Pendukung Traversal Token

Tiga fungsi pendukung menjadi inti dari traversal token. Fungsi currentToken() mengambil token pada posisi pembacaan; jika sudah melewati akhir, fungsi mengembalikan token END_OF_FILE. Fungsi peek(offset) melakukan lookahead tanpa memajukan pointer dan menjadi krusial untuk membedakan cabang produksi yang awalannya identik. Fungsi advance() memajukan indeks ke token berikutnya.

```
16  // Mengambil token yang sedang diproses saat ini
17  Token Parser::currentToken() {
18    if(this->currentTokenIndex < this->tokens.size()) {
19      return tokens[this->currentTokenIndex];
20    }
21    return Token(TokenType::END_OF_FILE, "EOF");
22  }
23
24  // Mengintip token di depan tanpa memajukan indeks
25  Token Parser::peek(int offset) {
26    if ((this->currentTokenIndex + offset) < this->tokens.size()) {
27      return tokens[this->currentTokenIndex+offset];
28    }
29    return Token(TokenType::END_OF_FILE, "EOF");
30  }
31
32  // Memajukan indeks ke token selanjutnya
33  void Parser::advance() {
34    if (this->currentTokenIndex < this->tokens.size()) {
35      this->currentTokenIndex++;
36    }
37  }
```

d. Fungsi match()

Fungsi match(expectedType) adalah jantung Parser. Bila tipe token saat ini cocok, fungsi membungkusnya menjadi leaf node (dengan menyimpan value untuk token bertipe IDENT/INTCON/REALCON/CHARCON/STRING) lalu memajukan

pointer; bila tidak cocok, fungsi memanggil reportError() dan mengembalikan nullptr.

```
40  ParseTreeNode* Parser::match(TokenType expectedType) {
41    Token current = this->currentToken();
42
43    Lexer dummyLexer("");
44    string currentStr = dummyLexer.tokenTypeToString(current.type);
45    string expectedStr = dummyLexer.tokenTypeToString(expectedType);
46
47    if (current.type == expectedType) {
48      string valToStore = "";
49      if (current.type == TokenType::IDENT || current.type == TokenType::INTCON ||
50        current.type == TokenType::REALCON || current.type == TokenType::CHARCON ||
51        current.type == TokenType::STRING) {
52        valToStore = current.value;
53      }
54      ParseTreeNode* node = new ParseTreeNode(currentStr, valToStore);
55      advance();
56      return node;
57    } else {
58      reportError(expectedStr, current.value);
59      return nullptr;
60    }
61  }
```

e. Penanganan Error Tanpa Crash

Setelah reportError() dipanggil, flag hasError diset true sehingga semua fungsi parse*() yang memeriksanya akan melakukan early return. Hal ini menjamin pesan error pertama tetap tunggal dan informatif sementara pointer dereference tidak terjadi.

```
64  void Parser::reportError(string expected, string found) {
65    if (!hasError) {
66      cout << "Syntax error: expected '" << expected << "', but found '" << found << "'" << endl
67      hasError = true;
68    }
69  }
```

f. Fungsi parseProgram()

Fungsi parseProgram() menjadi root dari parse tree dan menggambarkan struktur grammar tertinggi: program-header, declaration-part, compound-statement, lalu period.

```
6   ParseTreeNode* Parser::parseProgram() {
7       ParseTreeNode* node = new ParseTreeNode("<program>");
8
9       node->children.push_back(parseProgramHeader());
10      node->children.push_back(parseDeclarationPart());
11      node->children.push_back(parseCompoundStatement());
12      node->children.push_back(match(TokenType::PERIOD));
13
14      return node;
15  }
```

g. Pembedaan Assignment vs Procedure/Function Call

Keduanya sama-sama diawali oleh IDENT sehingga ambigu pada level statement. Parser melakukan lookahead satu token: bila yang menyusul adalah BECOMES, LBRACK, atau PERIOD maka token tersebut bagian dari <variable> pada <assignment-statement>; selain itu diperlakukan sebagai pemanggilan prosedur/fungsi. Statement kosong (empty statement) ditangani pada cabang default sehingga kode seperti ;;; di dalam begin-end tetap diterima.

```
53      Token tok = currentToken();
54      switch (tok.type) {
55          case TokenType::IDENT: {
56              Token next = peek(1);
57              if (next.type == TokenType::BECOMES ||
58                  next.type == TokenType::LBRACK ||
59                  next.type == TokenType::PERIOD) {
60                  node->children.push_back(parseAssignmentStatement());
61              } else {
62                  node->children.push_back(parseProcedureFunctionCall());
63              }
64              break;
65          }
```

h. Hierarki Ekspresi

Hierarki ekspresi dikodekan secara natural melalui rantai produksi <expression> -> <simple-expression> -> <term> -> <factor> sehingga presedensi relasional, aditif, dan multiplikatif tercermin pada urutan node parse tree. Setiap level menggunakan while-loop untuk menangani ekspresi berantai (left-recursion yang dikonversi menjadi iterasi) sehingga ekspresi seperti a + b - c tidak menyebabkan rekursi tak hingga.

```
// Hierarki kedua, yaitu perkalian (TIMES, RDIV, IDIV, IMOD, ANDSY)
ParseTreeNode* Parser::parseTerm() {
  ParseTreeNode* node = new ParseTreeNode("<term>");

  node->children.push_back(parseFactor());
  if (hasError) return node;

  TokenType type = currentToken().type;
  // Looping untuk menangani ekspresi perkalian berantai
  while (!hasError && (type == TokenType::TIMES || type == TokenType::RDIV || type == TokenType::IDIV || type == TokenType::IMOD || type == TokenType::ANDSY)) {
    ParseTreeNode* opNode = new ParseTreeNode("<multiplicative-operator>");
    opNode->children.push_back(match(type));
    node->children.push_back(opNode);
    node->children.push_back(parseFactor());
    type = currentToken().type;
  }
  return node;
}
```

i. Lookahead pada parseFactor()

Token IDENT di posisi <factor> bisa berupa identifier biasa, akses array/record, atau pemanggilan fungsi. Parser melakukan peek(1) untuk memilih cabang yang tepat: LBRACK atau PERIOD memicu parseVariable(), LPARENT memicu parseProcedureFunctionCall(), dan selain itu cukup match(IDENT) biasa. Operator unary not juga ditangani di level ini melalui rekursi ke parseFactor() untuk mendukung not not (a > b).

```
  if (type == TokenType::IDENT) {
    Token next = peek(1);
    // Lookahead untuk membedakan Identifier biasa, Variabel (Array/Record), dan Pemanggilan Fungsi
    if (next.type == TokenType::LBRACK || next.type == TokenType::PERIOD) {
      node->children.push_back(parseVariable());
    } else if (next.type == TokenType::LPARENT) {
      node->children.push_back(parseProcedureFunctionCall());
    } else {
      node->children.push_back(match(TokenType::IDENT));
    }
  } else if (type == TokenType::INTCON || type == TokenType::REALCON || type == TokenType::CHARCON || type == TokenType::STRING) {
    node->children.push_back(match(type));
  } else if (type == TokenType::NOTSY) {
    node->children.push_back(match(TokenType::NOTSY));
    node->children.push_back(parseFactor());
```

j. Pencetakan Parse Tree Berformat

Fungsi printTree() merekursifkan pencetakan parse tree dengan karakter pembatas hierarki (├── untuk anak tengah, └── untuk anak terakhir, dan | untuk garis vertikal yang melanjutkan parent), menghasilkan output rapi baik di terminal maupun di file txt. Indentasi dirakit secara inkremental berdasarkan posisi setiap anak.

```
72  void Parser::printTree(ParseTreeNode* node, ofstream& outFile, string indent, bool isLast, bool isRoot) {
73    if (node == nullptr) return;
74
75    string marker ="";
76    if (!isRoot) {
77      marker = isLast ? "└──" : "├──";
78      outFile << indent << marker;
79    }
80
81    outFile << node->label;
82    if (node->value != "") {
83      outFile << " (" << node->value << ") ";
84    }
85    outFile << endl;
86
87    string newIndent = indent;
88    if (!isRoot) {
89      newIndent += isLast ? "   " : "|  ";
90    }
91
92    for (size_t i = 0; i < node->children.size(); i++) {
93      bool isChildLast = (i == node->children.size() - 1);
94      printTree(node->children[i], outFile, newIndent, isChildLast, false);
95    }
96  }
```

D. PENGUJIAN

No	Input

1	`program SemicolonFest;`
	`var`
	`  x: integer;`
	`begin`
	`  ;;;`
	`  if x > 0 then`
	`  else ;`
	
	`  while x < 10 do`
	`  begin`
	`  end;`
	
	`  case x of`
	`  1: ;`
	`  2: begin ; end;`
	`  end`
	`end.`

	Output
	`<program>`


├──<program-header>
│ ├──programsy
│ ├──ident (SemicolonFest)
│ └──semicolon
├──<declaration-part>
│ └──<var-declaration>
│ ├──varsy
│ ├──<identifier-list>
│ │ └──ident (x)
│ ├──colon
│ ├──<type>
│ │ └──ident (integer)
│ └──semicolon
├──<compound-statement>
│ ├──beginsy
│ ├──<statement-list>
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ │ └──<if-statement>
│ │ │ ├──ifsy
│ │ │ ├──<expression>
│ │ │ │ ├──<simple-expression>
│ │ │ │ │ └──<term>
│ │ │ │ │ └──<factor>
│ │ │ │ │ └──ident (x)
│ │ │ │ ├──<relational-operator>
│ │ │ │ │ └──gtr
│ │ │ │ └──<simple-expression>
│ │ │ │ └──<term>
│ │ │ │ └──<factor>
│ │ │ │ └──intcon (0)
│ │ │ ├──thensy
│ │ │ ├──<statement>
│ │ │ ├──elsesy
│ │ │ └──<statement>
│ │ ├──semicolon
│ │ ├──<statement>

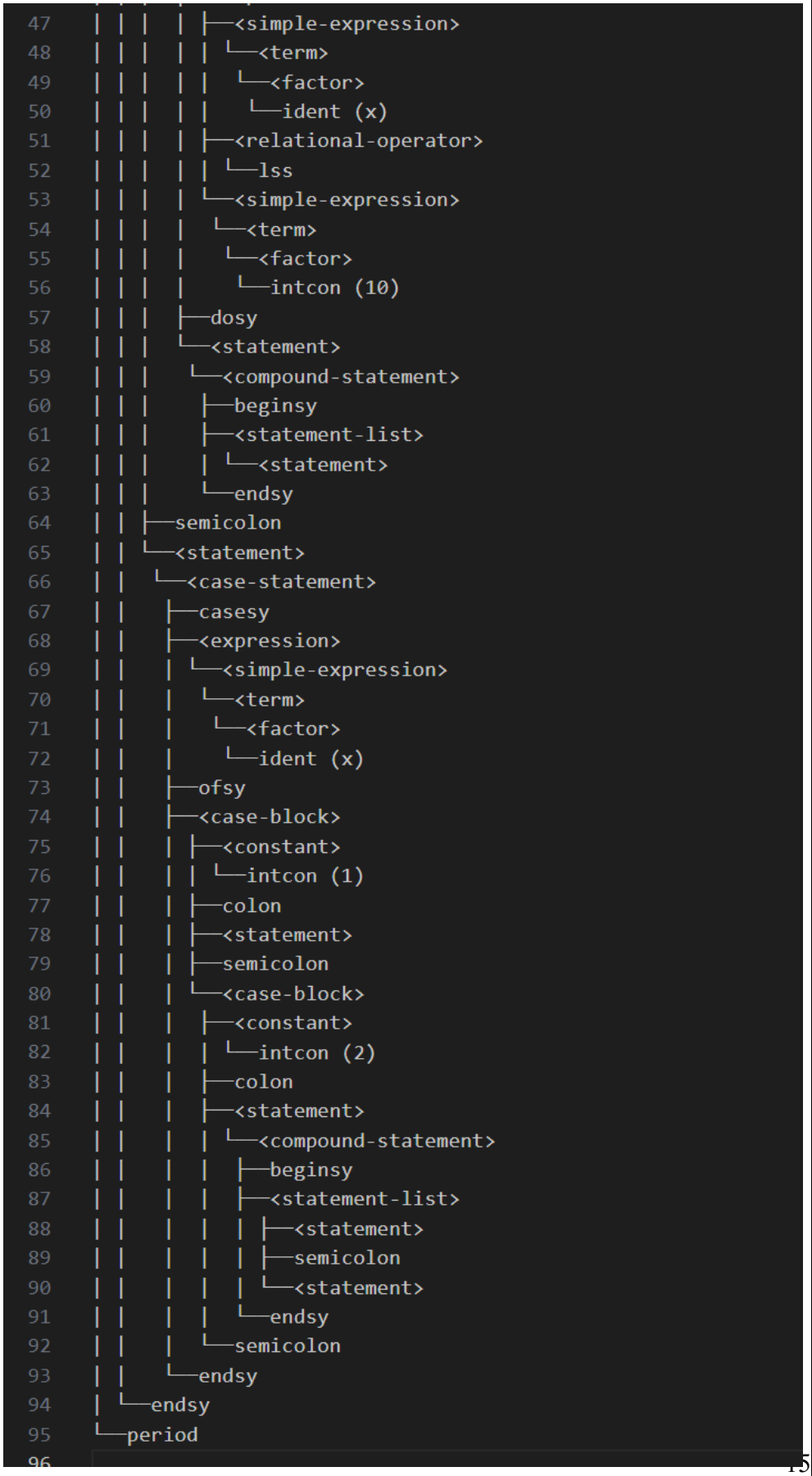
```
| | | └──<while-statement>
| | |  ├──whilesy
| | |  ├──<expression>
| | |  | ├──<simple-expression>
| | |  | | └──<term>
| | |  | |  └──<factor>
| | |  | |   └──ident (x)
| | |  | ├──<relational-operator>
| | |  | | └──lss
| | |  | └──<simple-expression>
| | |  |  └──<term>
| | |  |   └──<factor>
| | |  |    └──intcon (10)
| | |  ├──dosy
| | |  └──<statement>
| | |   └──<compound-statement>
| | |    ├──beginsy
| | |    ├──<statement-list>
| | |    | └──<statement>
| | |    └──endsy
| | ├──semicolon
| | └──<statement>
| |  └──<case-statement>
| |   ├──casesy
| |   ├──<expression>
| |   | └──<simple-expression>
| |   |  └──<term>
| |   |   └──<factor>
| |   |    └──ident (x)
| |   ├──ofsy
| |   ├──<case-block>
| |   | ├──<constant>
| |   | | └──intcon (1)
| |   | ├──colon
| |   | ├──<statement>
| |   | ├──semicolon
| |   | └──<case-block>
| |   |  ├──<constant>
| |   |  | └──intcon (2)
| |   |  ├──colon
| |   |  ├──<statement>
| |   |  | └──<compound-statement>
```

```
| |   |   |   ├──beginsy
| |   |   |   ├──<statement-list>
| |   |   |   | ├──<statement>
| |   |   |   | ├──semicolon
| |   |   |   | └──<statement>
| |   |   |   └──endsy
| |   |   └──semicolon
| |   └──endsy
| └──endsy
└──period
```

Screenshot Output

test > milestone-2 > output-1.txt

```
<program>
├──<program-header>
│ ├──programsy
│ ├──ident (SemicolonFest)
│ └──semicolon
├──<declaration-part>
│ └──<var-declaration>
│  ├──varsy
│  ├──<identifier-list>
│  │ └──ident (x)
│  ├──colon
│  ├──<type>
│  │ └──ident (integer)
│  └──semicolon
├──<compound-statement>
│ ├──beginsy
│ ├──<statement-list>
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ │ └──<if-statement>
│ │ │  ├──ifsy
│ │ │  ├──<expression>
│ │ │  │ ├──<simple-expression>
│ │ │  │ │ └──<term>
│ │ │  │ │  └──<factor>
│ │ │  │ │   └──ident (x)
│ │ │  │ ├──<relational-operator>
│ │ │  │ │ └──gtr
│ │ │  │ └──<simple-expression>
│ │ │  │  └──<term>
│ │ │  │   └──<factor>
│ │ │  │    └──intcon (0)
│ │ │  ├──thensy
│ │ │  ├──<statement>
│ │ │  ├──elsesy
│ │ │  └──<statement>
│ │ ├──semicolon
│ │ ├──<statement>
│ │ │ └──<while-statement>
│ │ │  ├──whilesy
│ │ │  ├──<expression>
│ │ │  │ ├──<simple-expression>
```


47 | | | | ├─<simple-expression>
48 | | | | | └─<term>
49 | | | | | └─<factor>
50 | | | | | └─ident (x)
51 | | | | ├─<relational-operator>
52 | | | | | └─lss
53 | | | | └─<simple-expression>
54 | | | | └─<term>
55 | | | | └─<factor>
56 | | | | └─intcon (10)
57 | | | ├─dosy
58 | | | └─<statement>
59 | | | └─<compound-statement>
60 | | | ├─beginsy
61 | | | ├─<statement-list>
62 | | | | └─<statement>
63 | | | └─endsy
64 | | ├─semicolon
65 | | └─<statement>
66 | | └─<case-statement>
67 | | ├─casesy
68 | | ├─<expression>
69 | | | └─<simple-expression>
70 | | | └─<term>
71 | | | └─<factor>
72 | | | └─ident (x)
73 | | ├─ofsy
74 | | ├─<case-block>
75 | | | ├─<constant>
76 | | | | └─intcon (1)
77 | | | ├─colon
78 | | | ├─<statement>
79 | | | ├─semicolon
80 | | | └─<case-block>
81 | | | ├─<constant>
82 | | | | └─intcon (2)
83 | | | ├─colon
84 | | | ├─<statement>
85 | | | | └─<compound-statement>
86 | | | | ├─beginsy
87 | | | | ├─<statement-list>
88 | | | | | ├─<statement>
89 | | | | | ├─semicolon
90 | | | | | └─<statement>
91 | | | | └─endsy
92 | | | └─semicolon
93 | | └─endsy
94 | └─endsy
95 └─period
96

2	Input

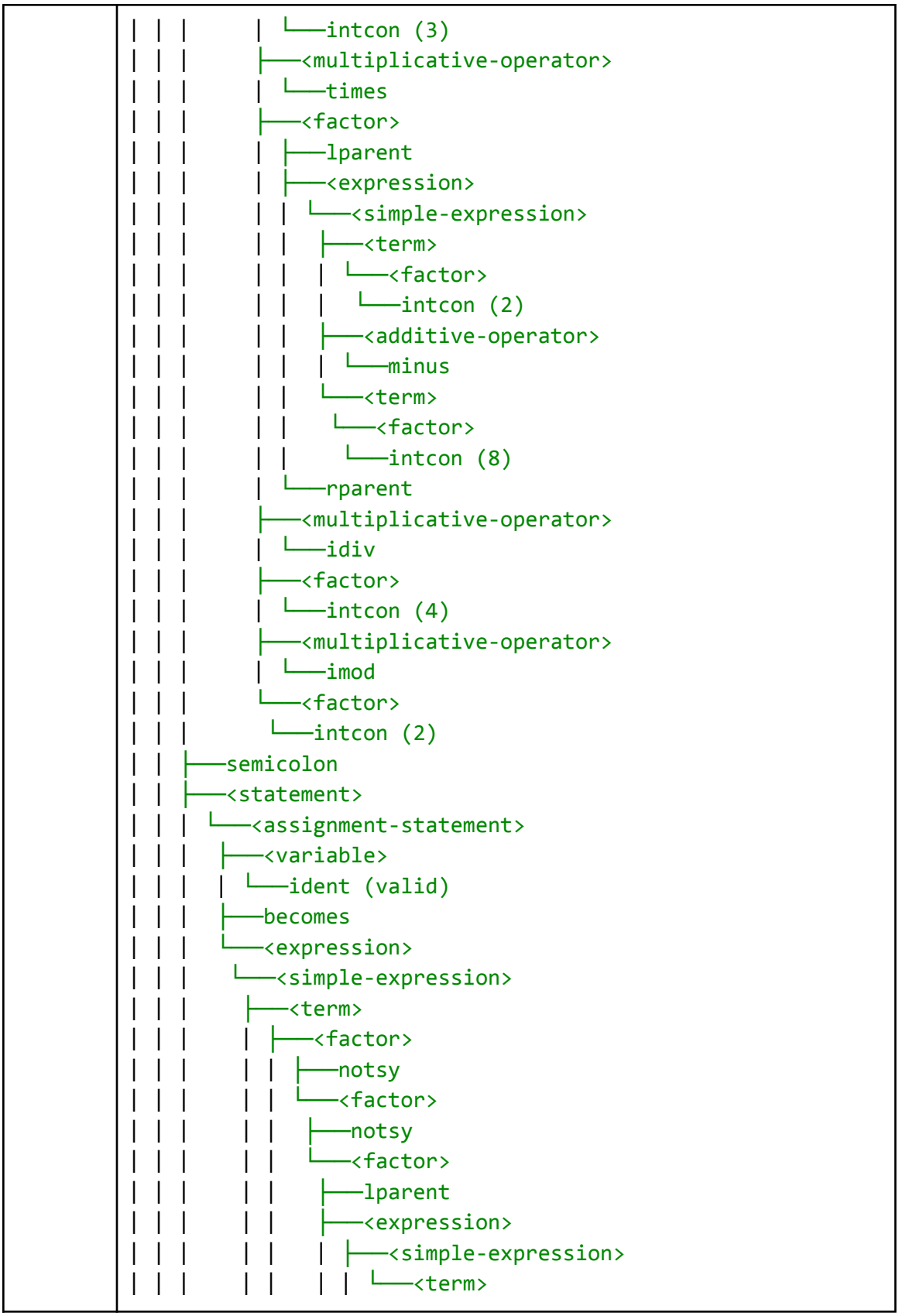
```
program UjiParser;
const
  MAKS == 100;
type
  TipeArray == array[1..10] of integer;
var
  a, b, x, y: integer;
  arr: TipeArray;
  valid: boolean;
begin
  (* EDGE CASE 1: Matematika Brutal & Unary Bertumpuk *)
  a := -5 + 3 * (2 - 8) div 4 mod 2;
  valid := not not (a > b) and (x == 5) or (MAKS <= 100);

  (* EDGE CASE 2: Dangling Else & Statement Kosong (Sesuai
Revisi QNA) *)
  if a > b then
    if x < 5 then
      a := 1
    else
      b := 2;
  ;

  (* EDGE CASE 3: Array Multi-Dimensi & Record Chaining di
Matematika *)
  x := fungsiSakti(a, b) * arr[1, x].data + 10;

  (* EDGE CASE 4: Case-Of dengan Banyak Konstanta Koma *)
  case arr[1] of
    1, 2, 3: x := 10;
    'a', 'b':
      begin
        x := 20;
        y := 30
      end;
  end;

  (* EDGE CASE 5: Loop Inception (For di dalam While di
dalam Repeat) *)
  repeat
    while x < 10 do
```

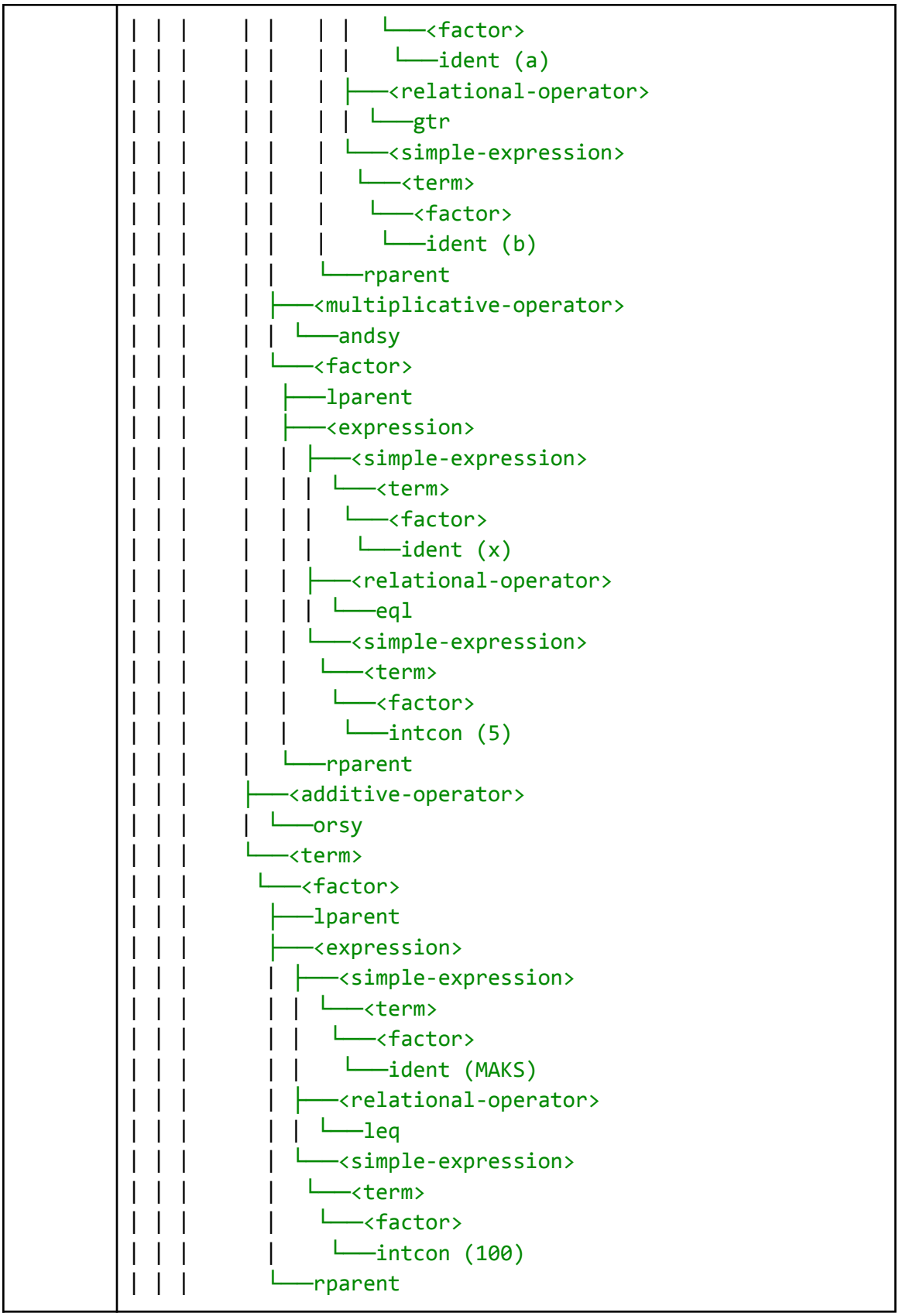
```
      for a := 1 to MAKS do
        x := x + 1
  until (x >= 100);

end.
```

Output

```
<program>
├──<program-header>
│ ├──programsy
│ ├──ident (UjiParser)
│ └──semicolon
├──<declaration-part>
│ ├──<const-declaration>
│ │ ├──constsy
│ │ ├──ident (MAKS)
│ │ ├──eql
│ │ ├──<constant>
│ │ │ └──intcon (100)
│ │ └──semicolon
│ ├──<type-declaration>
│ │ ├──typesy
│ │ ├──ident (TipeArray)
│ │ ├──eql
│ │ ├──<type>
│ │ │ └──<array-type>
│ │ │  ├──arraysy
│ │ │  ├──lbrack
│ │ │  ├──<range>
│ │ │  │ ├──<constant>
│ │ │  │ │ └──intcon (1)
│ │ │  │ ├──period
│ │ │  │ ├──period
│ │ │  │ └──<constant>
│ │ │  │  └──intcon (10)
│ │ │  ├──rbrack
│ │ │  ├──ofsy
│ │ │  └──<type>
│ │ │   └──ident (integer)
│ │ └──semicolon
│ └──<var-declaration>
│  ├──varsy
```

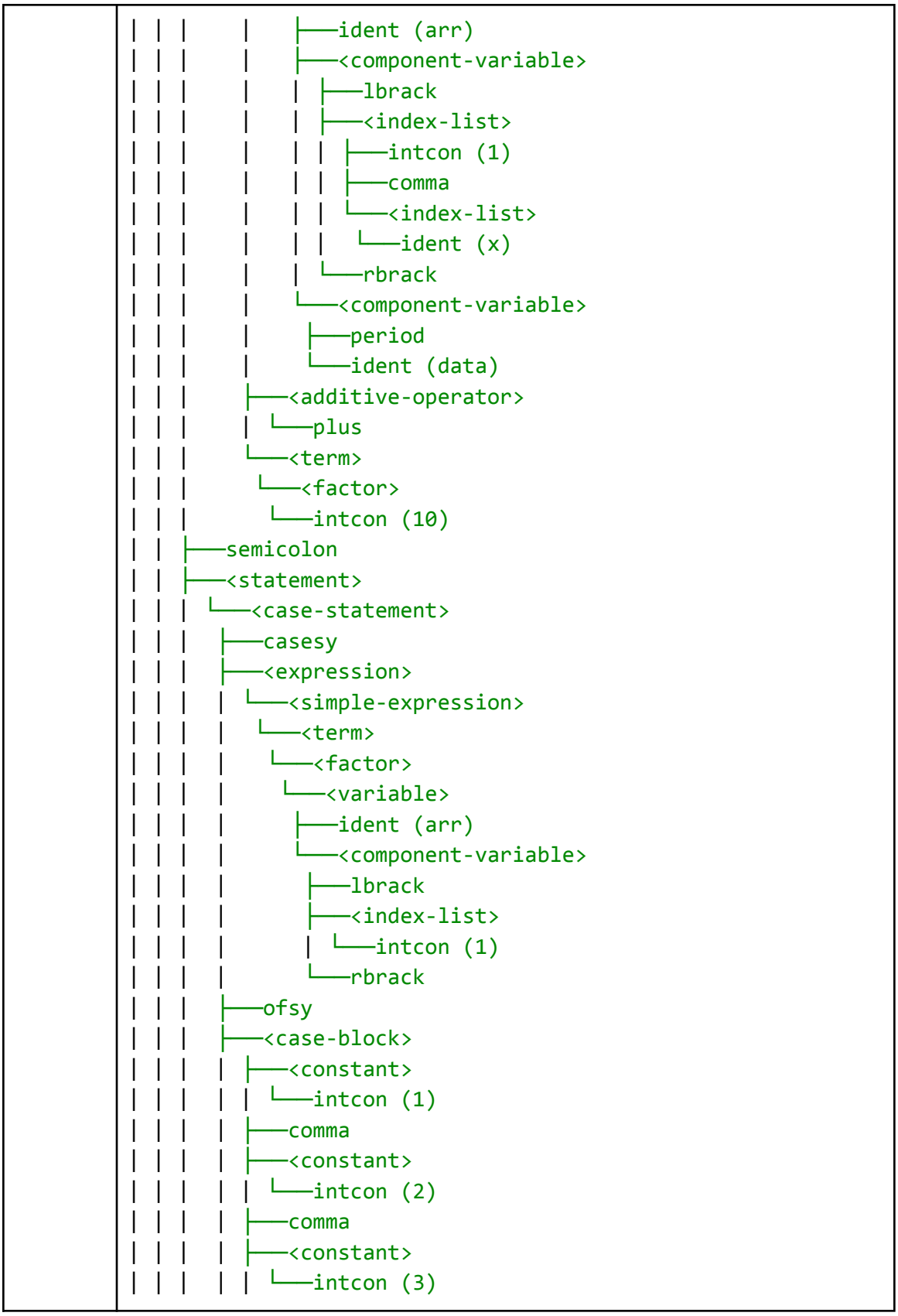
```
|   ├──<identifier-list>
|   | ├──ident (a)
|   | ├──comma
|   | ├──ident (b)
|   | ├──comma
|   | ├──ident (x)
|   | ├──comma
|   | └──ident (y)
|   ├──colon
|   ├──<type>
|   | └──ident (integer)
|   ├──semicolon
|   ├──<identifier-list>
|   | └──ident (arr)
|   ├──colon
|   ├──<type>
|   | └──ident (TipeArray)
|   ├──semicolon
|   ├──<identifier-list>
|   | └──ident (valid)
|   ├──colon
|   ├──<type>
|   | └──ident (boolean)
|   └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |   ├──<variable>
| | |   | └──ident (a)
| | |   ├──becomes
| | |   └──<expression>
| | |    └──<simple-expression>
| | |     ├──minus
| | |     ├──<term>
| | |     | └──<factor>
| | |     |  └──intcon (5)
| | |     ├──<additive-operator>
| | |     | └──plus
| | |     └──<term>
| | |      ├──<factor>
```


| | | | └──intcon (3)
| | | ├──<multiplicative-operator>
| | | | └──times
| | | ├──<factor>
| | | | ├──lparent
| | | | ├──<expression>
| | | | | └──<simple-expression>
| | | | | ├──<term>
| | | | | | └──<factor>
| | | | | | └──intcon (2)
| | | | | ├──<additive-operator>
| | | | | | └──minus
| | | | | └──<term>
| | | | | └──<factor>
| | | | | └──intcon (8)
| | | | └──rparent
| | | ├──<multiplicative-operator>
| | | | └──idiv
| | | ├──<factor>
| | | | └──intcon (4)
| | | ├──<multiplicative-operator>
| | | | └──imod
| | | └──<factor>
| | | └──intcon (2)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (valid)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | ├──<term>
| | | | ├──<factor>
| | | | | ├──notsy
| | | | | └──<factor>
| | | | | ├──notsy
| | | | | └──<factor>
| | | | | ├──lparent
| | | | | ├──<expression>
| | | | | | ├──<simple-expression>
| | | | | | | └──<term>


| | | | | | | └──<factor>
| | | | | | | └──ident (a)
| | | | | | ├──<relational-operator>
| | | | | | | └──gtr
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (b)
| | | | | └──rparent
| | | | | ├──<multiplicative-operator>
| | | | | | └──andsy
| | | | | └──<factor>
| | | | | ├──lparent
| | | | | ├──<expression>
| | | | | | ├──<simple-expression>
| | | | | | | └──<term>
| | | | | | | └──<factor>
| | | | | | | └──ident (x)
| | | | | | ├──<relational-operator>
| | | | | | | └──eql
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──intcon (5)
| | | | | └──rparent
| | | | ├──<additive-operator>
| | | | | └──orsy
| | | | └──<term>
| | | | └──<factor>
| | | | ├──lparent
| | | | ├──<expression>
| | | | | ├──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (MAKS)
| | | | | ├──<relational-operator>
| | | | | | └──leq
| | | | | └──<simple-expression>
| | | | | └──<term>
| | | | | └──<factor>
| | | | | └──intcon (100)
| | | | └──rparent

```
| | ├──semicolon
| | ├──<statement>
| | | └──<if-statement>
| | |  ├──ifsy
| | |  ├──<expression>
| | |  | ├──<simple-expression>
| | |  | | └──<term>
| | |  | |  └──<factor>
| | |  | |   └──ident (a)
| | |  | ├──<relational-operator>
| | |  | | └──gtr
| | |  | └──<simple-expression>
| | |  |  └──<term>
| | |  |   └──<factor>
| | |  |    └──ident (b)
| | |  ├──thensy
| | |  └──<statement>
| | |   └──<if-statement>
| | |    ├──ifsy
| | |    ├──<expression>
| | |    | ├──<simple-expression>
| | |    | | └──<term>
| | |    | |  └──<factor>
| | |    | |   └──ident (x)
| | |    | ├──<relational-operator>
| | |    | | └──lss
| | |    | └──<simple-expression>
| | |    |  └──<term>
| | |    |   └──<factor>
| | |    |    └──intcon (5)
| | |    ├──thensy
| | |    ├──<statement>
| | |    | └──<assignment-statement>
| | |    |  ├──<variable>
| | |    |  | └──ident (a)
| | |    |  ├──becomes
| | |    |  └──<expression>
| | |    |   └──<simple-expression>
| | |    |    └──<term>
| | |    |     └──<factor>
| | |    |      └──intcon (1)
| | |    ├──elsesy
```


| | | └──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (b)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | └──<term>
| | | └──<factor>
| | | └──intcon (2)
| | ├──semicolon
| | ├──<statement>
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (x)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | ├──<term>
| | | | ├──<factor>
| | | | | └──<procedure/function-call>
| | | | | ├──ident (fungsiSakti)
| | | | | ├──lparent
| | | | | ├──<parameter-list>
| | | | | | ├──<expression>
| | | | | | | └──<simple-expression>
| | | | | | | └──<term>
| | | | | | | └──<factor>
| | | | | | | └──ident (a)
| | | | | | ├──comma
| | | | | | └──<expression>
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (b)
| | | | | └──rparent
| | | | ├──<multiplicative-operator>
| | | | | └──times
| | | | └──<factor>
| | | | └──<variable>


| | | | | ├──ident (arr)
| | | | | ├──<component-variable>
| | | | | | ├──lbrack
| | | | | | ├──<index-list>
| | | | | | | ├──intcon (1)
| | | | | | | ├──comma
| | | | | | | └──<index-list>
| | | | | | | └──ident (x)
| | | | | | └──rbrack
| | | | | └──<component-variable>
| | | | | ├──period
| | | | | └──ident (data)
| | | | ├──<additive-operator>
| | | | | └──plus
| | | | └──<term>
| | | | └──<factor>
| | | | └──intcon (10)
| | | ├──semicolon
| | | ├──<statement>
| | | | └──<case-statement>
| | | | ├──casesy
| | | | ├──<expression>
| | | | | └──<simple-expression>
| | | | | └──<term>
| | | | | └──<factor>
| | | | | └──<variable>
| | | | | ├──ident (arr)
| | | | | └──<component-variable>
| | | | | ├──lbrack
| | | | | ├──<index-list>
| | | | | | └──intcon (1)
| | | | | └──rbrack
| | | | ├──ofsy
| | | | ├──<case-block>
| | | | | ├──<constant>
| | | | | | └──intcon (1)
| | | | | ├──comma
| | | | | ├──<constant>
| | | | | | └──intcon (2)
| | | | | ├──comma
| | | | | ├──<constant>
| | | | | | └──intcon (3)

```
| | |   | ├──colon
| | |   | ├──<statement>
| | |   | | └──<assignment-statement>
| | |   | |   ├──<variable>
| | |   | |   | └──ident (x)
| | |   | |   ├──becomes
| | |   | |   └──<expression>
| | |   | |    └──<simple-expression>
| | |   | |     └──<term>
| | |   | |      └──<factor>
| | |   | |       └──intcon (10)
| | |   | ├──semicolon
| | |   | └──<case-block>
| | |   |   ├──<constant>
| | |   |   | └──charcon ('a')
| | |   |   ├──comma
| | |   |   ├──<constant>
| | |   |   | └──charcon ('b')
| | |   |   ├──colon
| | |   |   ├──<statement>
| | |   |   | └──<compound-statement>
| | |   |   |   ├──beginsy
| | |   |   |   ├──<statement-list>
| | |   |   |   | ├──<statement>
| | |   |   |   | | └──<assignment-statement>
| | |   |   |   | |   ├──<variable>
| | |   |   |   | |   | └──ident (x)
| | |   |   |   | |   ├──becomes
| | |   |   |   | |   └──<expression>
| | |   |   |   | |    └──<simple-expression>
| | |   |   |   | |     └──<term>
| | |   |   |   | |      └──<factor>
| | |   |   |   | |       └──intcon (20)
| | |   |   |   | ├──semicolon
| | |   |   |   | └──<statement>
| | |   |   |   |   └──<assignment-statement>
| | |   |   |   |     ├──<variable>
| | |   |   |   |     | └──ident (y)
| | |   |   |   |     ├──becomes
| | |   |   |   |     └──<expression>
| | |   |   |   |      └──<simple-expression>
| | |   |   |   |       └──<term>
```


| | | | | | | └──<factor>
| | | | | | | └──intcon (30)
| | | | | | └──endsy
| | | | | └──semicolon
| | | | └──endsy
| | | ├──semicolon
| | | ├──<statement>
| | | | └──<repeat-statement>
| | | | ├──repeatsy
| | | | ├──<statement-list>
| | | | | └──<statement>
| | | | | └──<while-statement>
| | | | | ├──whilesy
| | | | | ├──<expression>
| | | | | | ├──<simple-expression>
| | | | | | | └──<term>
| | | | | | | └──<factor>
| | | | | | | └──ident (x)
| | | | | | ├──<relational-operator>
| | | | | | | └──lss
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──intcon (10)
| | | | | ├──dosy
| | | | | └──<statement>
| | | | | └──<for-statement>
| | | | | ├──forsy
| | | | | ├──ident (a)
| | | | | ├──becomes
| | | | | ├──<expression>
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──intcon (1)
| | | | | ├──tosy
| | | | | ├──<expression>
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (MAKS)
| | | | | ├──dosy

```
| | |   |      └──<statement>
| | |   |       └──<assignment-statement>
| | |   |        ├──<variable>
| | |   |        | └──ident (x)
| | |   |        ├──becomes
| | |   |        └──<expression>
| | |   |         └──<simple-expression>
| | |   |          ├──<term>
| | |   |          | └──<factor>
| | |   |          |  └──ident (x)
| | |   |          ├──<additive-operator>
| | |   |          | └──plus
| | |   |          └──<term>
| | |   |           └──<factor>
| | |   |            └──intcon (1)
| | |   ├──untilsy
| | |   └──<expression>
| | |    └──<simple-expression>
| | |     └──<term>
| | |      └──<factor>
| | |       ├──lparent
| | |       ├──<expression>
| | |       | ├──<simple-expression>
| | |       | | └──<term>
| | |       | |  └──<factor>
| | |       | |   └──ident (x)
| | |       | ├──<relational-operator>
| | |       | | └──geq
| | |       | └──<simple-expression>
| | |       |  └──<term>
| | |       |   └──<factor>
| | |       |    └──intcon (100)
| | |       └──rparent
| | ├──semicolon
| | └──<statement>
| └──endsy
└──period
```

Screenshot Output

test > milestone-2 > output-2.txt

```
<program>
├──<program-header>
| ├──programsy
| ├──ident (UjiParser)
| └──semicolon
├──<declaration-part>
| ├──<const-declaration>
| | ├──constsy
| | ├──ident (MAKS)
| | ├──eql
| | ├──<constant>
| | | └──intcon (100)
| | └──semicolon
| ├──<type-declaration>
| | ├──typesy
| | ├──ident (TipeArray)
| | ├──eql
| | ├──<type>
| | | └──<array-type>
| | |  ├──arraysy
| | |  ├──lbrack
| | |  ├──<range>
| | |  | ├──<constant>
| | |  | | └──intcon (1)
| | |  | ├──period
| | |  | ├──period
| | |  | └──<constant>
| | |  |  └──intcon (10)
| | |  ├──rbrack
| | |  ├──ofsy
| | |  └──<type>
| | |   └──ident (integer)
| | └──semicolon
| └──<var-declaration>
|  ├──varsy
|  ├──<identifier-list>
|  | ├──ident (a)
|  | ├──comma
|  | ├──ident (b)
|  | ├──comma
|  | ├──ident (x)
|  | ├──comma
|  | └──ident (y)
|  ├──colon
|  ├──<type>
|  | └──ident (integer)
|  ├──semicolon
|  ├──<identifier-list>
|  | └──ident (arr)
|  ├──colon
|  ├──<type>
|  | └──ident (TipeArray)
|  ├──semicolon
|  ├──<identifier-list>
|  | └──ident (valid)
|  ├──colon
|  ├──<type>
|  | └──ident (boolean)
```

test > milestone-2 > output-2.txt

```
50  |  ├──colon
51  |  ├──<type>
52  |  | └──ident (TipeArray)
53  |  ├──semicolon
54  |  ├──<identifier-list>
55  |  | └──ident (valid)
56  |  ├──colon
57  |  ├──<type>
58  |  | └──ident (boolean)
59  |  └──semicolon
60  ├──<compound-statement>
61  | ├──beginsy
62  | ├──<statement-list>
63  | | ├──<statement>
64  | | | └──<assignment-statement>
65  | | |  ├──<variable>
66  | | |  | └──ident (a)
67  | | |  ├──becomes
68  | | |  └──<expression>
69  | | |   └──<simple-expression>
70  | | |    ├──minus
71  | | |    ├──<term>
72  | | |    | └──<factor>
73  | | |    |  └──intcon (5)
74  | | |    ├──<additive-operator>
75  | | |    | └──plus
76  | | |    └──<term>
77  | | |     ├──<factor>
78  | | |     | └──intcon (3)
79  | | |     ├──<multiplicative-operator>
80  | | |     | └──times
81  | | |     ├──<factor>
82  | | |     | ├──lparent
83  | | |     | ├──<expression>
84  | | |     | | └──<simple-expression>
85  | | |     | |  ├──<term>
86  | | |     | |  | └──<factor>
87  | | |     | |  |  └──intcon (2)
88  | | |     | |  ├──<additive-operator>
89  | | |     | |  | └──minus
90  | | |     | |  └──<term>
91  | | |     | |   └──<factor>
92  | | |     | |    └──intcon (8)
93  | | |     | └──rparent
94  | | |     ├──<multiplicative-operator>
95  | | |     | └──idiv
96  | | |     ├──<factor>
97  | | |     | └──intcon (4)
98  | | |     ├──<multiplicative-operator>
99  | | |     | └──imod
100 | | |     └──<factor>
101 | | |      └──intcon (2)
102 | | ├──semicolon
103 | | ├──<statement>
104 | | | └──<assignment-statement>
105 | | |  ├──<variable>
106 | | |  | └──ident (valid)
```

test > milestone-2 > output-2.txt

```
106   | | |  | └──ident (valid)
107   | | |  ├──becomes
108   | | |  └──<expression>
109   | | |   └──<simple-expression>
110   | | |    ├──<term>
111   | | |    | ├──<factor>
112   | | |    | | ├──notsy
113   | | |    | | └──<factor>
114   | | |    | |  ├──notsy
115   | | |    | |  └──<factor>
116   | | |    | |   ├──lparent
117   | | |    | |   ├──<expression>
118   | | |    | |   | ├──<simple-expression>
119   | | |    | |   | | └──<term>
120   | | |    | |   | |  └──<factor>
121   | | |    | |   | |   └──ident (a)
122   | | |    | |   | ├──<relational-operator>
123   | | |    | |   | | └──gtr
124   | | |    | |   | └──<simple-expression>
125   | | |    | |   |  └──<term>
126   | | |    | |   |   └──<factor>
127   | | |    | |   |    └──ident (b)
128   | | |    | |   └──rparent
129   | | |    | ├──<multiplicative-operator>
130   | | |    | | └──andsy
131   | | |    | └──<factor>
132   | | |    |  ├──lparent
133   | | |    |  ├──<expression>
134   | | |    |  | ├──<simple-expression>
135   | | |    |  | | └──<term>
136   | | |    |  | |  └──<factor>
137   | | |    |  | |   └──ident (x)
138   | | |    |  | ├──<relational-operator>
139   | | |    |  | | └──eql
140   | | |    |  | └──<simple-expression>
141   | | |    |  |  └──<term>
142   | | |    |  |   └──<factor>
143   | | |    |  |    └──intcon (5)
144   | | |    |  └──rparent
145   | | |    ├──<additive-operator>
146   | | |    | └──orsy
147   | | |    └──<term>
148   | | |     └──<factor>
149   | | |      ├──lparent
150   | | |      ├──<expression>
151   | | |      | ├──<simple-expression>
152   | | |      | | └──<term>
153   | | |      | |  └──<factor>
154   | | |      | |   └──ident (MAKS)
155   | | |      | ├──<relational-operator>
156   | | |      | | └──leq
157   | | |      | └──<simple-expression>
158   | | |      |  └──<term>
159   | | |      |   └──<factor>
160   | | |      |    └──intcon (100)
161   | | |      └──rparent
162   | | ├──semicolon
```

test > milestone-2 > output-2.txt

```
162   | | ├──semicolon
163   | | ├──<statement>
164   | | | └──<if-statement>
165   | | |  ├──ifsy
166   | | |  ├──<expression>
167   | | |  | ├──<simple-expression>
168   | | |  | | └──<term>
169   | | |  | |  └──<factor>
170   | | |  | |   └──ident (a)
171   | | |  | ├──<relational-operator>
172   | | |  | | └──gtr
173   | | |  | └──<simple-expression>
174   | | |  |  └──<term>
175   | | |  |   └──<factor>
176   | | |  |    └──ident (b)
177   | | |  ├──thensy
178   | | |  └──<statement>
179   | | |   └──<if-statement>
180   | | |    ├──ifsy
181   | | |    ├──<expression>
182   | | |    | ├──<simple-expression>
183   | | |    | | └──<term>
184   | | |    | |  └──<factor>
185   | | |    | |   └──ident (x)
186   | | |    | ├──<relational-operator>
187   | | |    | | └──lss
188   | | |    | └──<simple-expression>
189   | | |    |  └──<term>
190   | | |    |   └──<factor>
191   | | |    |    └──intcon (5)
192   | | |    ├──thensy
193   | | |    ├──<statement>
194   | | |    | └──<assignment-statement>
195   | | |    |  ├──<variable>
196   | | |    |  | └──ident (a)
197   | | |    |  ├──becomes
198   | | |    |  └──<expression>
199   | | |    |   └──<simple-expression>
200   | | |    |    └──<term>
201   | | |    |     └──<factor>
202   | | |    |      └──intcon (1)
203   | | |    ├──elsesy
204   | | |    └──<statement>
205   | | |     └──<assignment-statement>
206   | | |      ├──<variable>
207   | | |      | └──ident (b)
208   | | |      ├──becomes
209   | | |      └──<expression>
210   | | |       └──<simple-expression>
211   | | |        └──<term>
212   | | |         └──<factor>
213   | | |          └──intcon (2)
214   | | ├──semicolon
215   | | ├──<statement>
216   | | ├──semicolon
217   | | ├──<statement>
218   | | | └──<assignment-statement>
```

test > milestone-2 > output-2.txt

```
218   | | | └─<assignment-statement>
219   | | |  ├─<variable>
220   | | |  | └─ident (x)
221   | | |  ├─becomes
222   | | |  └─<expression>
223   | | |   └─<simple-expression>
224   | | |    ├─<term>
225   | | |    | ├─<factor>
226   | | |    | | └─<procedure/function-call>
227   | | |    | |  ├─ident (fungsiSakti)
228   | | |    | |  ├─lparent
229   | | |    | |  ├─<parameter-list>
230   | | |    | |  | ├─<expression>
231   | | |    | |  | | └─<simple-expression>
232   | | |    | |  | |  └─<term>
233   | | |    | |  | |   └─<factor>
234   | | |    | |  | |    └─ident (a)
235   | | |    | |  | ├─comma
236   | | |    | |  | └─<expression>
237   | | |    | |  |  └─<simple-expression>
238   | | |    | |  |   └─<term>
239   | | |    | |  |    └─<factor>
240   | | |    | |  |     └─ident (b)
241   | | |    | |  └─rparent
242   | | |    | ├─<multiplicative-operator>
243   | | |    | | └─times
244   | | |    | └─<factor>
245   | | |    |  └─<variable>
246   | | |    |   ├─ident (arr)
247   | | |    |   ├─<component-variable>
248   | | |    |   | ├─lbrack
249   | | |    |   | ├─<index-list>
250   | | |    |   | | ├─intcon (1)
251   | | |    |   | | ├─comma
252   | | |    |   | | └─<index-list>
253   | | |    |   | |  └─ident (x)
254   | | |    |   | └─rbrack
255   | | |    |   └─<component-variable>
256   | | |    |    ├─period
257   | | |    |    └─ident (data)
258   | | |    ├─<additive-operator>
259   | | |    | └─plus
260   | | |    └─<term>
261   | | |     └─<factor>
262   | | |      └─intcon (10)
263   | | ├─semicolon
264   | | ├─<statement>
265   | | | └─<case-statement>
266   | | |  ├─casesy
267   | | |  ├─<expression>
268   | | |  | └─<simple-expression>
269   | | |  |  └─<term>
270   | | |  |   └─<factor>
271   | | |  |    └─<variable>
272   | | |  |     ├─ident (arr)
273   | | |  |     └─<component-variable>
274   | | |  |      ├─lbrack
```

test > milestone-2 > output-2.txt

```
274   | | | |       ├──lbrack
275   | | | |       ├──<index-list>
276   | | | |       | └──intcon (1)
277   | | | |       └──rbrack
278   | | | ├──ofsy
279   | | | ├──<case-block>
280   | | | | ├──<constant>
281   | | | | | └──intcon (1)
282   | | | | ├──comma
283   | | | | ├──<constant>
284   | | | | | └──intcon (2)
285   | | | | ├──comma
286   | | | | ├──<constant>
287   | | | | | └──intcon (3)
288   | | | | ├──colon
289   | | | | ├──<statement>
290   | | | | | └──<assignment-statement>
291   | | | | |  ├──<variable>
292   | | | | |  | └──ident (x)
293   | | | | |  ├──becomes
294   | | | | |  └──<expression>
295   | | | | |    └──<simple-expression>
296   | | | | |      └──<term>
297   | | | | |        └──<factor>
298   | | | | |          └──intcon (10)
299   | | | | ├──semicolon
300   | | | | └──<case-block>
301   | | | |  ├──<constant>
302   | | | |  | └──charcon ('a')
303   | | | |  ├──comma
304   | | | |  ├──<constant>
305   | | | |  | └──charcon ('b')
306   | | | |  ├──colon
307   | | | |  ├──<statement>
308   | | | |  | └──<compound-statement>
309   | | | |  |  ├──beginsy
310   | | | |  |  ├──<statement-list>
311   | | | |  |  | ├──<statement>
312   | | | |  |  | | └──<assignment-statement>
313   | | | |  |  | |  ├──<variable>
314   | | | |  |  | |  | └──ident (x)
315   | | | |  |  | |  ├──becomes
316   | | | |  |  | |  └──<expression>
317   | | | |  |  | |    └──<simple-expression>
318   | | | |  |  | |      └──<term>
319   | | | |  |  | |        └──<factor>
320   | | | |  |  | |          └──intcon (20)
321   | | | |  |  | ├──semicolon
322   | | | |  |  | └──<statement>
323   | | | |  |  |  └──<assignment-statement>
324   | | | |  |  |   ├──<variable>
325   | | | |  |  |   | └──ident (y)
326   | | | |  |  |   ├──becomes
327   | | | |  |  |   └──<expression>
328   | | | |  |  |     └──<simple-expression>
329   | | | |  |  |       └──<term>
330   | | | |  |  |         └──<factor>
```

test > milestone-2 > output-2.txt

```
330   | | | | | | |       └─<factor>
331   | | | | | | |         └─intcon (30)
332   | | | | | |  └─endsy
333   | | | | |  └─semicolon
334   | | | |  └─endsy
335   | | ├─semicolon
336   | | ├─<statement>
337   | | | └─<repeat-statement>
338   | | |  ├─repeatsy
339   | | |  ├─<statement-list>
340   | | |  | └─<statement>
341   | | |  |  └─<while-statement>
342   | | |  |   ├─whilesy
343   | | |  |   ├─<expression>
344   | | |  |   | ├─<simple-expression>
345   | | |  |   | | └─<term>
346   | | |  |   | |  └─<factor>
347   | | |  |   | |   └─ident (x)
348   | | |  |   | ├─<relational-operator>
349   | | |  |   | | └─lss
350   | | |  |   | └─<simple-expression>
351   | | |  |   |  └─<term>
352   | | |  |   |   └─<factor>
353   | | |  |   |    └─intcon (10)
354   | | |  |   ├─dosy
355   | | |  |   └─<statement>
356   | | |  |    └─<for-statement>
357   | | |  |     ├─forsy
358   | | |  |     ├─ident (a)
359   | | |  |     ├─becomes
360   | | |  |     ├─<expression>
361   | | |  |     | └─<simple-expression>
362   | | |  |     |  └─<term>
363   | | |  |     |   └─<factor>
364   | | |  |     |    └─intcon (1)
365   | | |  |     ├─tosy
366   | | |  |     ├─<expression>
367   | | |  |     | └─<simple-expression>
368   | | |  |     |  └─<term>
369   | | |  |     |   └─<factor>
370   | | |  |     |    └─ident (MAKS)
371   | | |  |     ├─dosy
372   | | |  |     └─<statement>
373   | | |  |      └─<assignment-statement>
374   | | |  |       ├─<variable>
375   | | |  |       | └─ident (x)
376   | | |  |       ├─becomes
377   | | |  |       └─<expression>
378   | | |  |        └─<simple-expression>
379   | | |  |         ├─<term>
380   | | |  |         | └─<factor>
381   | | |  |         |  └─ident (x)
382   | | |  |         ├─<additive-operator>
383   | | |  |         | └─plus
384   | | |  |         └─<term>
385   | | |  |          └─<factor>
386   | | |  |           └─intcon (1)
```

```
386   | | |  |           └─intcon (1)
387   | | |  ├─untilsy
388   | | |  └─<expression>
389   | | |   └─<simple-expression>
390   | | |    └─<term>
391   | | |     └─<factor>
392   | | |      ├─lparent
393   | | |      ├─<expression>
394   | | |      | ├─<simple-expression>
395   | | |      | | └─<term>
396   | | |      | |  └─<factor>
397   | | |      | |   └─ident (x)
398   | | |      | ├─<relational-operator>
399   | | |      | | └─geq
400   | | |      | └─<simple-expression>
401   | | |      |  └─<term>
402   | | |      |   └─<factor>
403   | | |      |    └─intcon (100)
404   | | |      └─rparent
405   | | ├─semicolon
406   | | └─<statement>
407   | └─endsy
408   └─period
409
```

3	**Input**

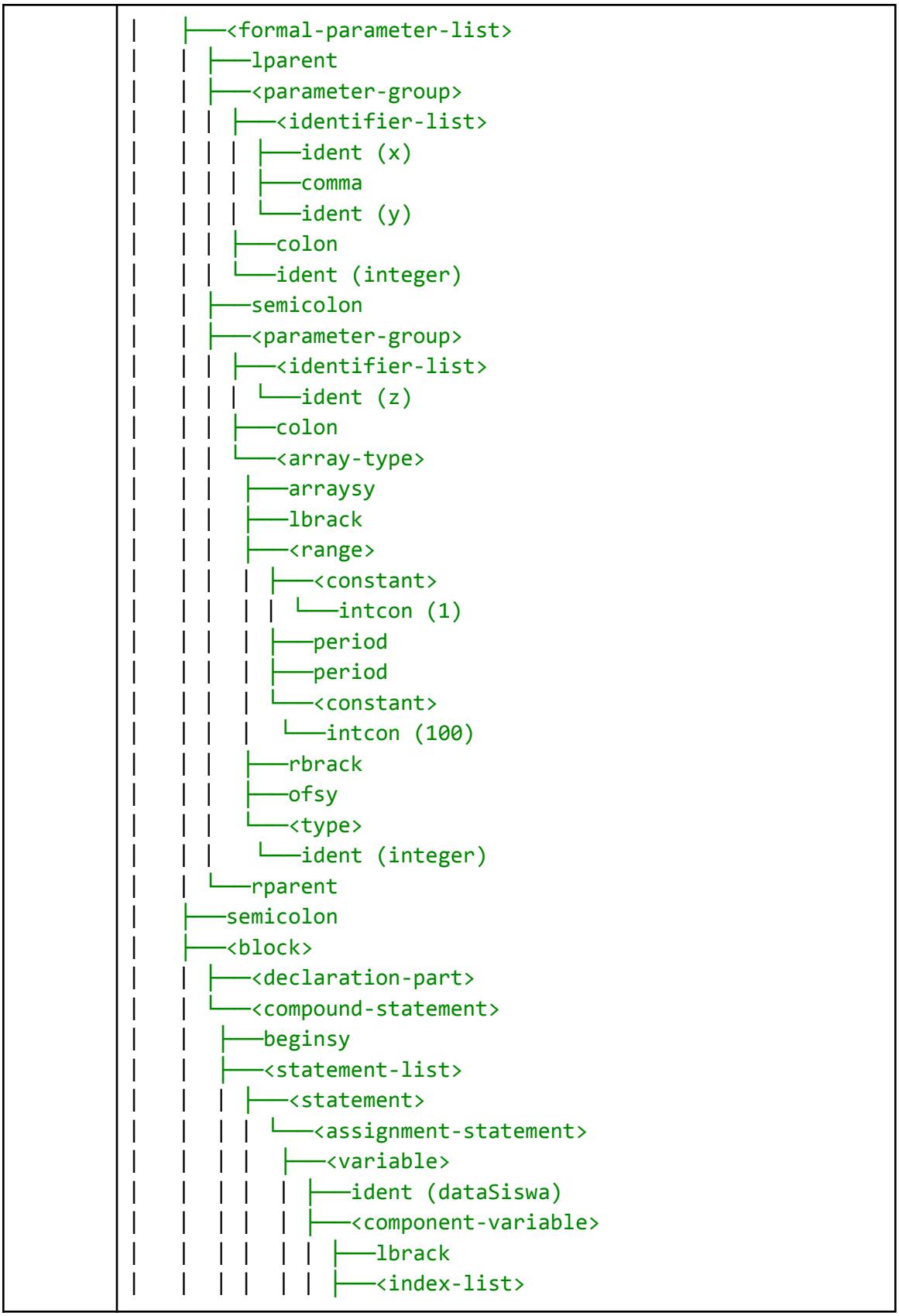
```
program DataInception;
type
  Bulan == (Jan, Feb, Mar);
  Siswa == record
    nilai: array[1..10] of integer;
    lulus: boolean
  end;
var
  dataSiswa: array['a'..'z'] of Siswa;
procedure prosesData(x, y: integer; z: array[1..100] of
integer);
begin
  dataSiswa['b'].nilai[5] := x + y * z[10];
  dataSiswa['c'].lulus := not (z[1] < 0)
end;
```

```
begin
  prosesData(1, 2, dataSiswa['a'].nilai)
end.
```

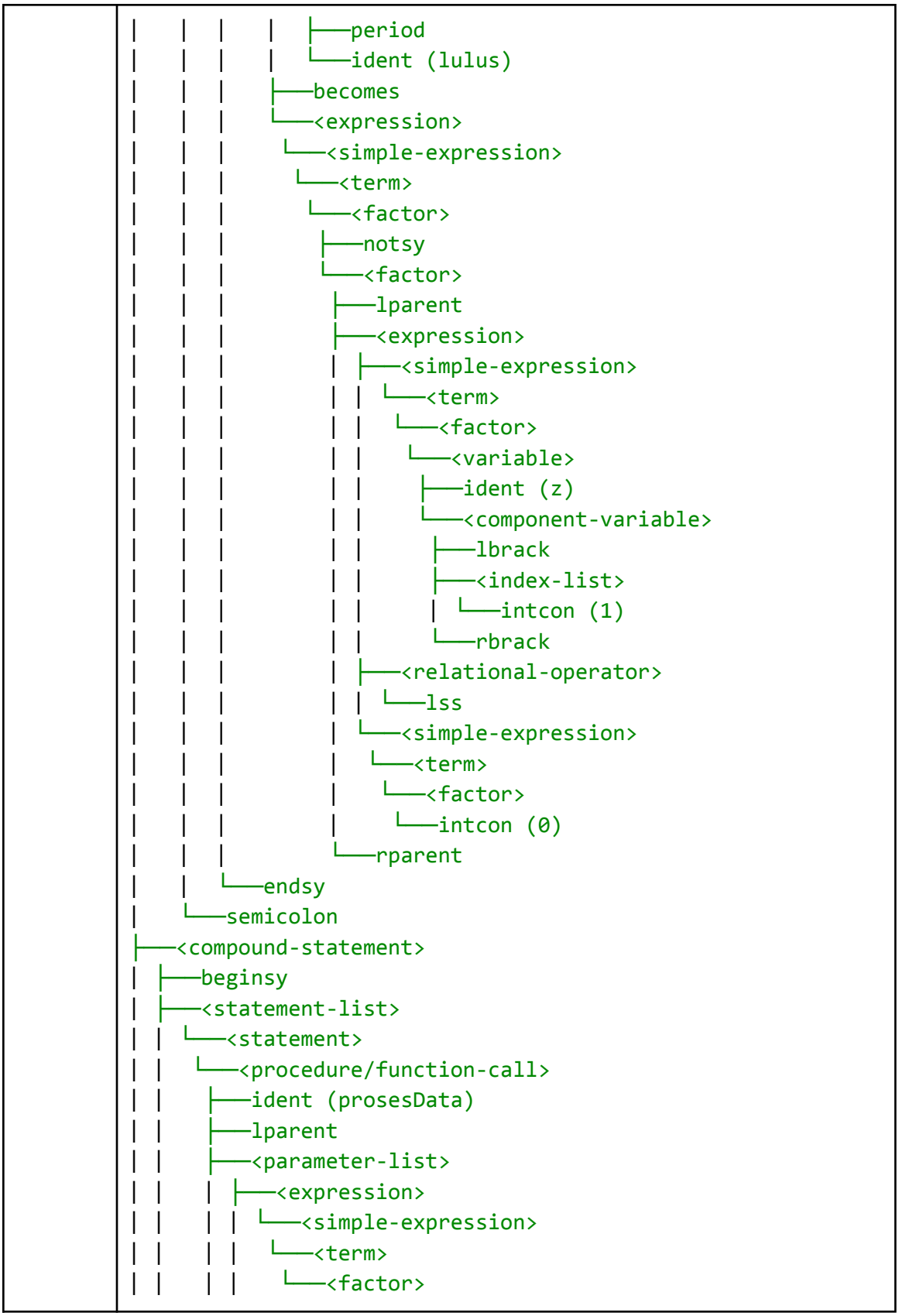
Output

```
<program>
├──<program-header>
│ ├──programsy
│ ├──ident (DataInception)
│ └──semicolon
├──<declaration-part>
│ ├──<type-declaration>
│ │ ├──typesy
│ │ ├──ident (Bulan)
│ │ ├──eql
│ │ ├──<type>
│ │ │ └──<enumerated>
│ │ │  ├──lparent
│ │ │  ├──ident (Jan)
│ │ │  ├──comma
│ │ │  ├──ident (Feb)
│ │ │  ├──comma
│ │ │  ├──ident (Mar)
│ │ │  └──rparent
│ │ ├──semicolon
│ │ ├──ident (Siswa)
│ │ ├──eql
│ │ ├──<type>
│ │ │ └──<record-type>
│ │ │  ├──recordsy
│ │ │  ├──<field-list>
│ │ │  │ ├──<field-part>
│ │ │  │ │ ├──<identifier-list>
│ │ │  │ │ │ └──ident (nilai)
│ │ │  │ │ ├──colon
│ │ │  │ │ └──<type>
│ │ │  │ │  └──<array-type>
│ │ │  │ │  ├──arraysy
│ │ │  │ │  ├──lbrack
│ │ │  │ │  ├──<range>
│ │ │  │ │  │ ├──<constant>
│ │ │  │ │  │ │ └──intcon (1)
```

```
| | |   | |    | ├──period
| | |   | |    | ├──period
| | |   | |    | └──<constant>
| | |   | |    |   └──intcon (10)
| | |   | |    ├──rbrack
| | |   | |    ├──ofsy
| | |   | |    └──<type>
| | |   | |      └──ident (integer)
| | |   | ├──semicolon
| | |   | └──<field-part>
| | |   |   ├──<identifier-list>
| | |   |   | └──ident (lulus)
| | |   |   ├──colon
| | |   |   └──<type>
| | |   |     └──ident (boolean)
| | |   └──endsy
| |  └──semicolon
| ├──<var-declaration>
| |  ├──varsy
| |  ├──<identifier-list>
| |  | └──ident (dataSiswa)
| |  ├──colon
| |  ├──<type>
| |  | └──<array-type>
| |  |   ├──arraysy
| |  |   ├──lbrack
| |  |   ├──<range>
| |  |   | ├──<constant>
| |  |   | | └──charcon ('a')
| |  |   | ├──period
| |  |   | ├──period
| |  |   | └──<constant>
| |  |   |   └──charcon ('z')
| |  |   ├──rbrack
| |  |   ├──ofsy
| |  |   └──<type>
| |  |     └──ident (Siswa)
| |  └──semicolon
| └──<subprogram-declaration>
|   └──<procedure-declaration>
|     ├──proceduresy
|     ├──ident (prosesData)
```


| ├──<formal-parameter-list>
| | ├──lparent
| | ├──<parameter-group>
| | | ├──<identifier-list>
| | | | ├──ident (x)
| | | | ├──comma
| | | | └──ident (y)
| | | ├──colon
| | | └──ident (integer)
| | ├──semicolon
| | ├──<parameter-group>
| | | ├──<identifier-list>
| | | | └──ident (z)
| | | ├──colon
| | | └──<array-type>
| | | ├──arraysy
| | | ├──lbrack
| | | ├──<range>
| | | | ├──<constant>
| | | | | └──intcon (1)
| | | | ├──period
| | | | ├──period
| | | | └──<constant>
| | | | └──intcon (100)
| | | ├──rbrack
| | | ├──ofsy
| | | └──<type>
| | | └──ident (integer)
| | └──rparent
| ├──semicolon
| ├──<block>
| | ├──<declaration-part>
| | └──<compound-statement>
| | ├──beginsy
| | ├──<statement-list>
| | | ├──<statement>
| | | | └──<assignment-statement>
| | | | ├──<variable>
| | | | | ├──ident (dataSiswa)
| | | | | ├──<component-variable>
| | | | | | ├──lbrack
| | | | | | ├──<index-list>

```
|   |   | |   | | | └──charcon ('b')
|   |   | |   | | └──rbrack
|   |   | |   | ├──<component-variable>
|   |   | |   | | ├──period
|   |   | |   | | └──ident (nilai)
|   |   | |   | └──<component-variable>
|   |   | |   |   ├──lbrack
|   |   | |   |   ├──<index-list>
|   |   | |   |   | └──intcon (5)
|   |   | |   |   └──rbrack
|   |   | |   ├──becomes
|   |   | |   └──<expression>
|   |   | |    └──<simple-expression>
|   |   | |     ├──<term>
|   |   | |     | └──<factor>
|   |   | |     |  └──ident (x)
|   |   | |     ├──<additive-operator>
|   |   | |     | └──plus
|   |   | |     └──<term>
|   |   | |      ├──<factor>
|   |   | |      | └──ident (y)
|   |   | |      ├──<multiplicative-operator>
|   |   | |      | └──times
|   |   | |      └──<factor>
|   |   | |       └──<variable>
|   |   | |        ├──ident (z)
|   |   | |        └──<component-variable>
|   |   | |         ├──lbrack
|   |   | |         ├──<index-list>
|   |   | |         | └──intcon (10)
|   |   | |         └──rbrack
|   |   | ├──semicolon
|   |   | └──<statement>
|   |   |  └──<assignment-statement>
|   |   |   ├──<variable>
|   |   |   | ├──ident (dataSiswa)
|   |   |   | ├──<component-variable>
|   |   |   | | ├──lbrack
|   |   |   | | ├──<index-list>
|   |   |   | | | └──charcon ('c')
|   |   |   | | └──rbrack
|   |   |   | └──<component-variable>
```


| | | | ├──period
| | | | └──ident (lulus)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | └──<term>
| | | └──<factor>
| | | ├──notsy
| | | └──<factor>
| | | ├──lparent
| | | ├──<expression>
| | | | ├──<simple-expression>
| | | | | └──<term>
| | | | | └──<factor>
| | | | | └──<variable>
| | | | | ├──ident (z)
| | | | | └──<component-variable>
| | | | | ├──lbrack
| | | | | ├──<index-list>
| | | | | | └──intcon (1)
| | | | | └──rbrack
| | | | ├──<relational-operator>
| | | | | └──lss
| | | | └──<simple-expression>
| | | | └──<term>
| | | | └──<factor>
| | | | └──intcon (0)
| | | └──rparent
| | └──endsy
| └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | └──<statement>
| | └──<procedure/function-call>
| | ├──ident (prosesData)
| | ├──lparent
| | ├──<parameter-list>
| | | ├──<expression>
| | | | └──<simple-expression>
| | | | └──<term>
| | | | └──<factor>

```
| |   | |     └──intcon (1)
| |   | ├──comma
| |   | ├──<expression>
| |   | | └──<simple-expression>
| |   | |  └──<term>
| |   | |   └──<factor>
| |   | |    └──intcon (2)
| |   | ├──comma
| |   | └──<expression>
| |   |  └──<simple-expression>
| |   |   └──<term>
| |   |    └──<factor>
| |   |     └──<variable>
| |   |      ├──ident (dataSiswa)
| |   |      ├──<component-variable>
| |   |      | ├──lbrack
| |   |      | ├──<index-list>
| |   |      | | └──charcon ('a')
| |   |      | └──rbrack
| |   |      └──<component-variable>
| |   |       ├──period
| |   |       └──ident (nilai)
| |   └──rparent
| └──endsy
└──period
```

Screenshot Output

test > milestone-2 > output-3.txt

```
<program>
├──<program-header>
| ├──programsy
| ├──ident (DataInception)
| └──semicolon
├──<declaration-part>
| ├──<type-declaration>
| | ├──typesy
| | ├──ident (Bulan)
| | ├──eql
| | ├──<type>
| | | └──<enumerated>
| | |  ├──lparent
| | |  ├──ident (Jan)
| | |  ├──comma
| | |  ├──ident (Feb)
| | |  ├──comma
| | |  ├──ident (Mar)
| | |  └──rparent
| | ├──semicolon
| | ├──ident (Siswa)
| | ├──eql
| | ├──<type>
| | | └──<record-type>
| | |  ├──recordsy
| | |  ├──<field-list>
| | |  | ├──<field-part>
| | |  | | ├──<identifier-list>
| | |  | | | └──ident (nilai)
| | |  | | ├──colon
| | |  | | └──<type>
| | |  | |  └──<array-type>
| | |  | |  ├──arraysy
| | |  | |  ├──lbrack
| | |  | |  ├──<range>
| | |  | |  | ├──<constant>
| | |  | |  | | └──intcon (1)
| | |  | |  | ├──period
| | |  | |  | ├──period
| | |  | |  | └──<constant>
| | |  | |  |  └──intcon (10)
| | |  | |  ├──rbrack
| | |  | |  ├──ofsy
| | |  | |  └──<type>
| | |  | |  └──ident (integer)
| | |  | ├──semicolon
| | |  | └──<field-part>
| | |  |  ├──<identifier-list>
| | |  |  | └──ident (lulus)
| | |  |  ├──colon
| | |  |  └──<type>
```

test > milestone-2 > output-3.txt

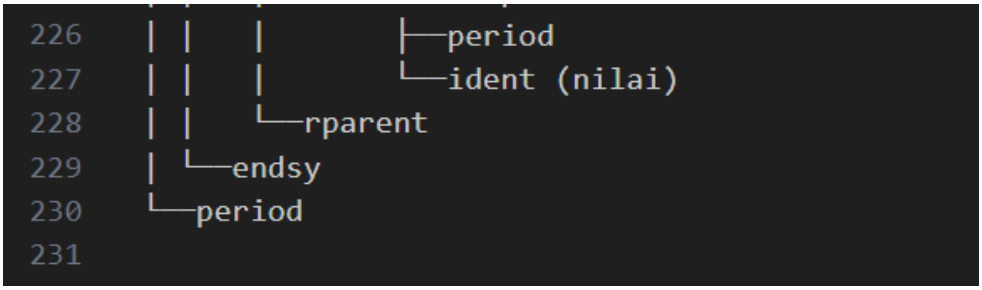
```
58  | | | └─ident (dataSiswa)
59  | | ├─colon
60  | | ├─<type>
61  | | | └─<array-type>
62  | | |   ├─arraysy
63  | | |   ├─lbrack
64  | | |   ├─<range>
65  | | |   | ├─<constant>
66  | | |   | | └─charcon ('a')
67  | | |   | ├─period
68  | | |   | ├─period
69  | | |   | └─<constant>
70  | | |   |   └─charcon ('z')
71  | | |   ├─rbrack
72  | | |   ├─ofsy
73  | | |   └─<type>
74  | | |     └─ident (Siswa)
75  | | └─semicolon
76  | └─<subprogram-declaration>
77  |   └─<procedure-declaration>
78  |     ├─proceduresy
79  |     ├─ident (prosesData)
80  |     ├─<formal-parameter-list>
81  |     | ├─lparent
82  |     | ├─<parameter-group>
83  |     | | ├─<identifier-list>
84  |     | | | ├─ident (x)
85  |     | | | ├─comma
86  |     | | | └─ident (y)
87  |     | | ├─colon
88  |     | | └─ident (integer)
89  |     | ├─semicolon
90  |     | ├─<parameter-group>
91  |     | | ├─<identifier-list>
92  |     | | | └─ident (z)
93  |     | | ├─colon
94  |     | | └─<array-type>
95  |     | |   ├─arraysy
96  |     | |   ├─lbrack
97  |     | |   ├─<range>
98  |     | |   | ├─<constant>
99  |     | |   | | └─intcon (1)
100 |     | |   | ├─period
101 |     | |   | ├─period
102 |     | |   | └─<constant>
103 |     | |   |   └─intcon (100)
104 |     | |   ├─rbrack
105 |     | |   ├─ofsy
106 |     | |   └─<type>
107 |     | |     └─ident (integer)
108 |     | └─rparent
109 |     ├─semicolon
```

test > milestone-2 > output-3.txt

```
114   |   |   ├──<statement-list>
115   |   |   | ├──<statement>
116   |   |   | | └──<assignment-statement>
117   |   |   | |   ├──<variable>
118   |   |   | |   | ├──ident (dataSiswa)
119   |   |   | |   | ├──<component-variable>
120   |   |   | |   | | ├──lbrack
121   |   |   | |   | | ├──<index-list>
122   |   |   | |   | | | └──charcon ('b')
123   |   |   | |   | | └──rbrack
124   |   |   | |   | ├──<component-variable>
125   |   |   | |   | | ├──period
126   |   |   | |   | | └──ident (nilai)
127   |   |   | |   | └──<component-variable>
128   |   |   | |   |   ├──lbrack
129   |   |   | |   |   ├──<index-list>
130   |   |   | |   |   | └──intcon (5)
131   |   |   | |   |   └──rbrack
132   |   |   | |   ├──becomes
133   |   |   | |   └──<expression>
134   |   |   | |     └──<simple-expression>
135   |   |   | |       ├──<term>
136   |   |   | |       | └──<factor>
137   |   |   | |       |   └──ident (x)
138   |   |   | |       ├──<additive-operator>
139   |   |   | |       | └──plus
140   |   |   | |       └──<term>
141   |   |   | |         ├──<factor>
142   |   |   | |         | └──ident (y)
143   |   |   | |         ├──<multiplicative-operator>
144   |   |   | |         | └──times
145   |   |   | |         └──<factor>
146   |   |   | |           └──<variable>
147   |   |   | |             ├──ident (z)
148   |   |   | |             └──<component-variable>
149   |   |   | |               ├──lbrack
150   |   |   | |               ├──<index-list>
151   |   |   | |               | └──intcon (10)
152   |   |   | |               └──rbrack
153   |   |   | ├──semicolon
154   |   |   | └──<statement>
155   |   |   |   └──<assignment-statement>
156   |   |   |     ├──<variable>
157   |   |   |     | ├──ident (dataSiswa)
158   |   |   |     | ├──<component-variable>
159   |   |   |     | | ├──lbrack
160   |   |   |     | | ├──<index-list>
161   |   |   |     | | | └──charcon ('c')
162   |   |   |     | | └──rbrack
163   |   |   |     | └──<component-variable>
164   |   |   |     |   ├──period
165   |   |   |     |   └──ident (lulus)
166   |   |   |     ├──becomes
167   |   |   |     └──<expression>
168   |   |   |       └──<simple-expression>
169   |   |   |         └──<term>
170   |   |   |           └──<factor>
```

test > milestone-2 > output-3.txt

```
170   |   |   |       └─<factor>
171   |   |   |        ├─notsy
172   |   |   |        └─<factor>
173   |   |   |         ├─lparent
174   |   |   |         ├─<expression>
175   |   |   |         | ├─<simple-expression>
176   |   |   |         | | └─<term>
177   |   |   |         | |  └─<factor>
178   |   |   |         | |   └─<variable>
179   |   |   |         | |    ├─ident (z)
180   |   |   |         | |    └─<component-variable>
181   |   |   |         | |     ├─lbrack
182   |   |   |         | |     ├─<index-list>
183   |   |   |         | |     | └─intcon (1)
184   |   |   |         | |     └─rbrack
185   |   |   |         | ├─<relational-operator>
186   |   |   |         | | └─lss
187   |   |   |         | └─<simple-expression>
188   |   |   |         |  └─<term>
189   |   |   |         |   └─<factor>
190   |   |   |         |    └─intcon (0)
191   |   |   |         └─rparent
192   |   |   └─endsy
193   |   └─semicolon
194   ├─<compound-statement>
195   | ├─beginsy
196   | ├─<statement-list>
197   | | └─<statement>
198   | |  └─<procedure/function-call>
199   | |   ├─ident (prosesData)
200   | |   ├─lparent
201   | |   ├─<parameter-list>
202   | |   | ├─<expression>
203   | |   | | └─<simple-expression>
204   | |   | |  └─<term>
205   | |   | |   └─<factor>
206   | |   | |    └─intcon (1)
207   | |   | ├─comma
208   | |   | ├─<expression>
209   | |   | | └─<simple-expression>
210   | |   | |  └─<term>
211   | |   | |   └─<factor>
212   | |   | |    └─intcon (2)
213   | |   | ├─comma
214   | |   | └─<expression>
215   | |   |  └─<simple-expression>
216   | |   |   └─<term>
217   | |   |    └─<factor>
218   | |   |     └─<variable>
219   | |   |      ├─ident (dataSiswa)
220   | |   |      ├─<component-variable>
221   | |   |      | ├─lbrack
222   | |   |      | ├─<index-list>
223   | |   |      | | └─charcon ('a')
224   | |   |      | └─rbrack
225   | |   |      └─<component-variable>
226   | |   |       ├─period
```



4

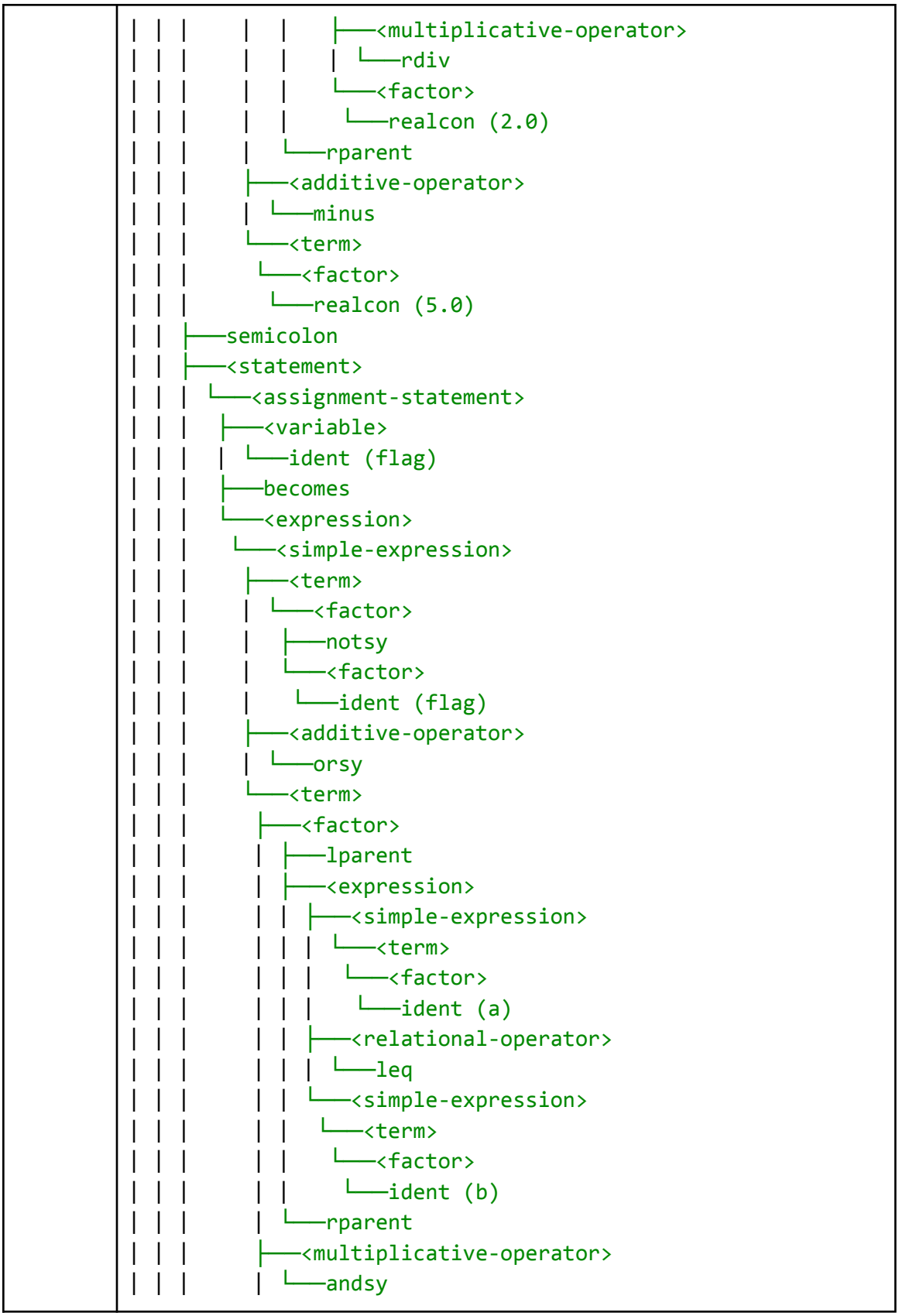
Input

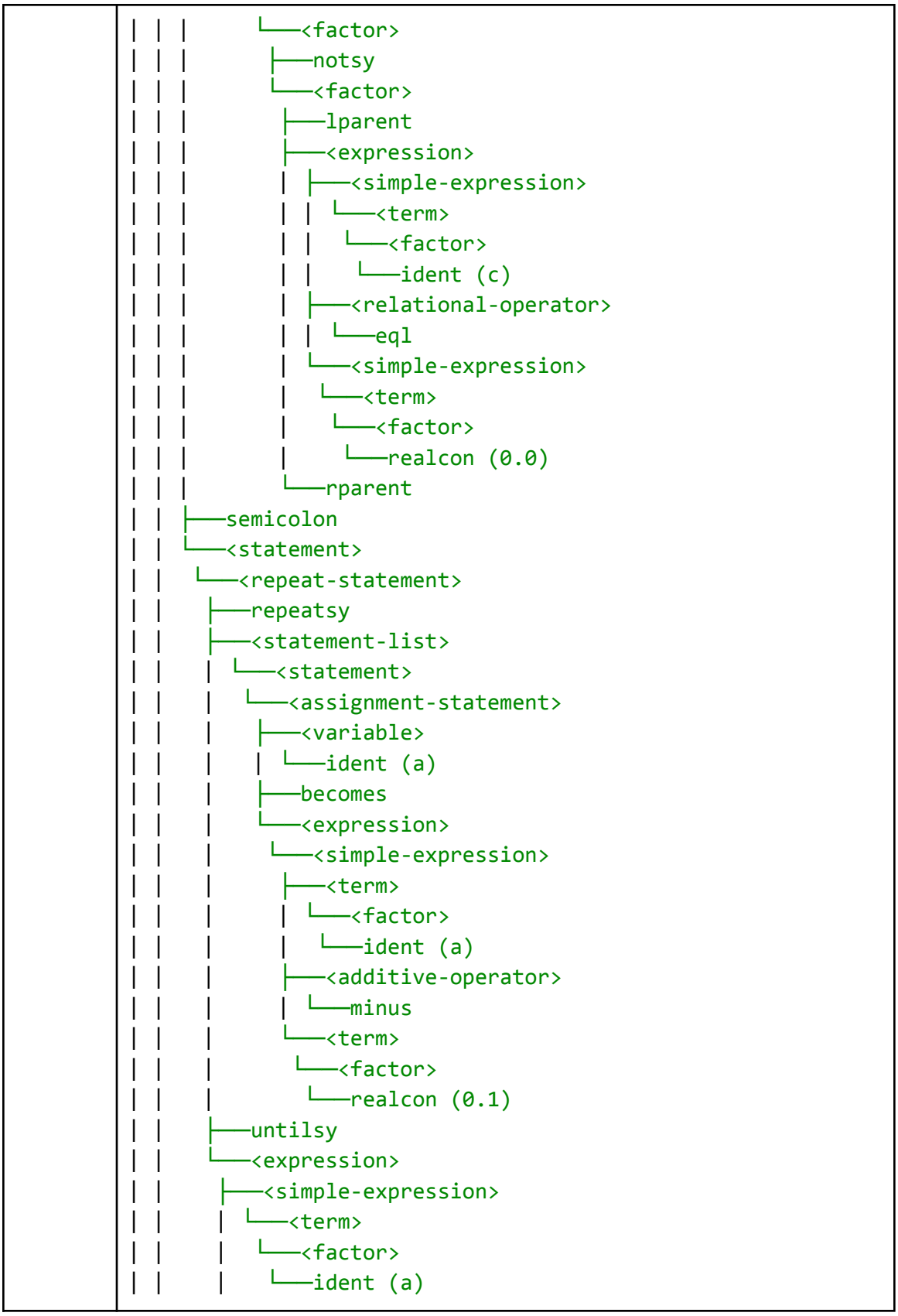
```
program Math;
const
  PI == 3.14;
var
  a, b, c: real;
  flag: boolean;
begin
  a := -PI + 2.5 * (10.0 / 2.0) - 5.0;
  flag := not flag or (a <= b) and not (c == 0.0);
  repeat
    a := a - 0.1
  until a < 0.0
end.
```

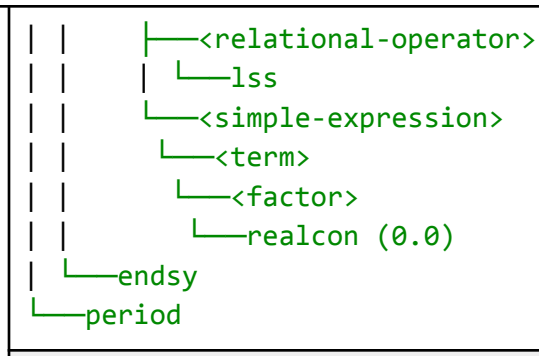
Output

```
<program>
├──<program-header>
│  ├──programsy
│  ├──ident (Math)
│  └──semicolon
├──<declaration-part>
│  ├──<const-declaration>
│  │  ├──constsy
│  │  ├──ident (PI)
│  │  ├──eql
│  │  ├──<constant>
│  │  │  └──realcon (3.14)
│  │  └──semicolon
│  └──<var-declaration>
│     ├──varsy
│     ├──<identifier-list>
│     │  ├──ident (a)
```

```
|   | ├──comma
|   | ├──ident (b)
|   | ├──comma
|   | └──ident (c)
|   ├──colon
|   ├──<type>
|   | └──ident (real)
|   ├──semicolon
|   ├──<identifier-list>
|   | └──ident (flag)
|   ├──colon
|   ├──<type>
|   | └──ident (boolean)
|   └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |   ├──<variable>
| | |   | └──ident (a)
| | |   ├──becomes
| | |   └──<expression>
| | |    └──<simple-expression>
| | |     ├──minus
| | |     ├──<term>
| | |     | └──<factor>
| | |     |  └──ident (PI)
| | |     ├──<additive-operator>
| | |     | └──plus
| | |     ├──<term>
| | |     | ├──<factor>
| | |     | | └──realcon (2.5)
| | |     | ├──<multiplicative-operator>
| | |     | | └──times
| | |     | └──<factor>
| | |     |  ├──lparent
| | |     |  ├──<expression>
| | |     |  | └──<simple-expression>
| | |     |  |  └──<term>
| | |     |  |   ├──<factor>
| | |     |  |   | └──realcon (10.0)
```


| | | | | | ├──<multiplicative-operator>
| | | | | | | └──rdiv
| | | | | | └──<factor>
| | | | | | └──realcon (2.0)
| | | | | └──rparent
| | | | ├──<additive-operator>
| | | | | └──minus
| | | | └──<term>
| | | | └──<factor>
| | | | └──realcon (5.0)
| | | ├──semicolon
| | | ├──<statement>
| | | | └──<assignment-statement>
| | | | ├──<variable>
| | | | | └──ident (flag)
| | | | ├──becomes
| | | | └──<expression>
| | | | └──<simple-expression>
| | | | ├──<term>
| | | | | └──<factor>
| | | | | ├──notsy
| | | | | └──<factor>
| | | | | └──ident (flag)
| | | | ├──<additive-operator>
| | | | | └──orsy
| | | | └──<term>
| | | | ├──<factor>
| | | | | ├──lparent
| | | | | ├──<expression>
| | | | | | ├──<simple-expression>
| | | | | | | └──<term>
| | | | | | | └──<factor>
| | | | | | | └──ident (a)
| | | | | | ├──<relational-operator>
| | | | | | | └──leq
| | | | | | └──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (b)
| | | | | └──rparent
| | | | ├──<multiplicative-operator>
| | | | | └──andsy

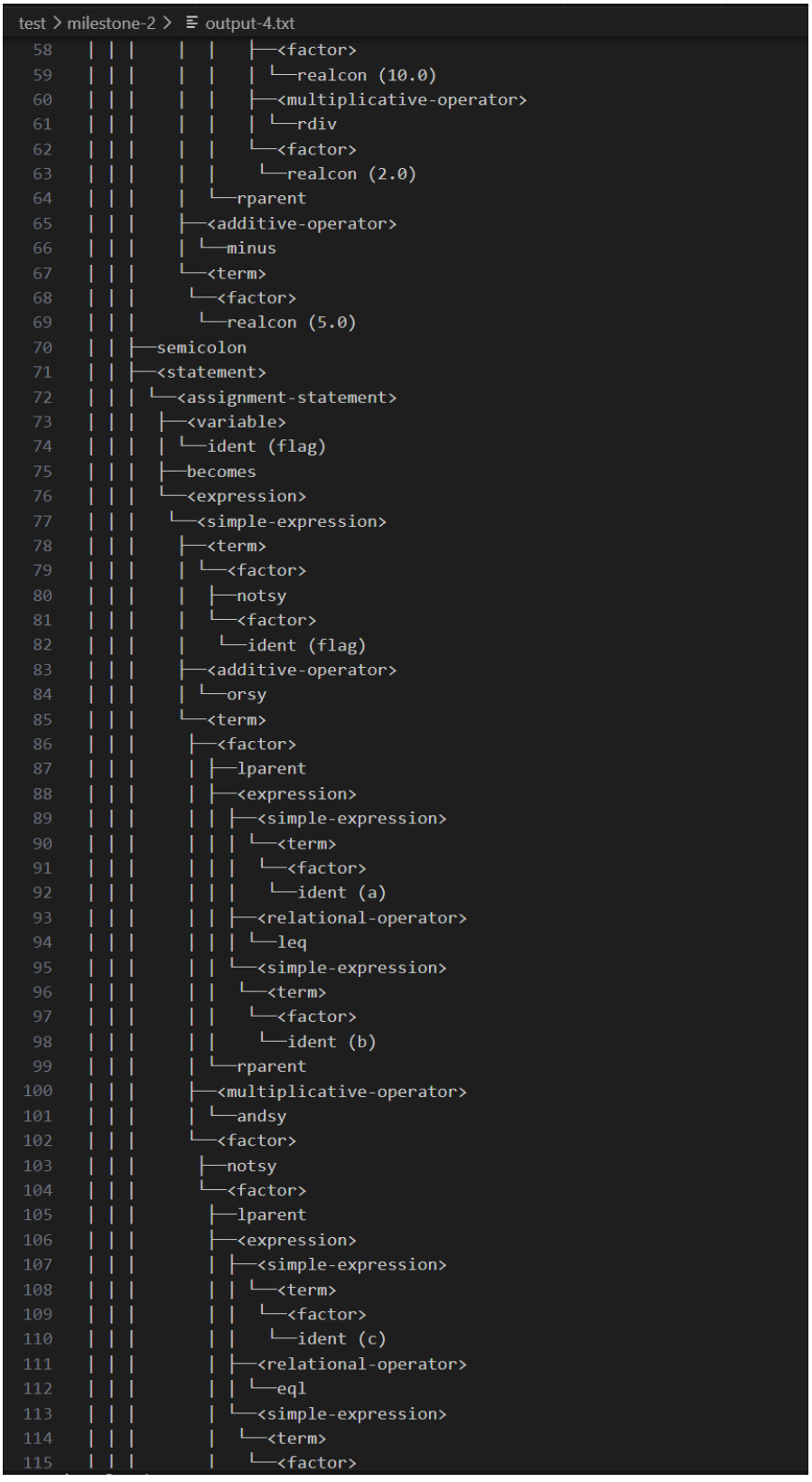

└──<factor>
├──notsy
└──<factor>
├──lparent
├──<expression>
├──<simple-expression>
└──<term>
└──<factor>
└──ident (c)
├──<relational-operator>
└──eql
└──<simple-expression>
└──<term>
└──<factor>
└──realcon (0.0)
└──rparent
├──semicolon
└──<statement>
└──<repeat-statement>
├──repeatsy
├──<statement-list>
└──<statement>
└──<assignment-statement>
├──<variable>
└──ident (a)
├──becomes
└──<expression>
└──<simple-expression>
├──<term>
└──<factor>
└──ident (a)
├──<additive-operator>
└──minus
└──<term>
└──<factor>
└──realcon (0.1)
├──untilsy
└──<expression>
├──<simple-expression>
└──<term>
└──<factor>
└──ident (a)

	
	Screenshot Output

test > milestone-2 > output-4.txt

```
<program>
├──<program-header>
│ ├──programsy
│ ├──ident (Math)
│ └──semicolon
├──<declaration-part>
│ ├──<const-declaration>
│ │ ├──constsy
│ │ ├──ident (PI)
│ │ ├──eql
│ │ ├──<constant>
│ │ │ └──realcon (3.14)
│ │ └──semicolon
│ └──<var-declaration>
│  ├──varsy
│  ├──<identifier-list>
│  │ ├──ident (a)
│  │ ├──comma
│  │ ├──ident (b)
│  │ ├──comma
│  │ └──ident (c)
│  ├──colon
│  ├──<type>
│  │ └──ident (real)
│  ├──semicolon
│  ├──<identifier-list>
│  │ └──ident (flag)
│  ├──colon
│  ├──<type>
│  │ └──ident (boolean)
│  └──semicolon
├──<compound-statement>
│ ├──beginsy
│ ├──<statement-list>
│ │ ├──<statement>
│ │ │ └──<assignment-statement>
│ │ │  ├──<variable>
│ │ │  │ └──ident (a)
│ │ │  ├──becomes
│ │ │  └──<expression>
│ │ │  └──<simple-expression>
│ │ │  ├──minus
│ │ │  ├──<term>
│ │ │  │ └──<factor>
│ │ │  │  └──ident (PI)
│ │ │  ├──<additive-operator>
│ │ │  │ └──plus
│ │ │  ├──<term>
│ │ │  │ ├──<factor>
│ │ │  │ │ └──realcon (2.5)
│ │ │  │ ├──<multiplicative-operator>
│ │ │  │ │ └──times
│ │ │  │ └──<factor>
│ │ │  │  ├──lparent
│ │ │  │  ├──<expression>
│ │ │  │  │ └──<simple-expression>
│ │ │  │  │  └──<term>
│ │ │  │  │  ├──<factor>
```

test > milestone-2 > output-4.txt



```
58  | | |    |  |   ├─<factor>
59  | | |    |  |   | └─realcon (10.0)
60  | | |    |  |   ├─<multiplicative-operator>
61  | | |    |  |   | └─rdiv
62  | | |    |  |   └─<factor>
63  | | |    |  |     └─realcon (2.0)
64  | | |    |  └─rparent
65  | | |    ├─<additive-operator>
66  | | |    | └─minus
67  | | |    └─<term>
68  | | |     └─<factor>
69  | | |       └─realcon (5.0)
70  | | ├─semicolon
71  | | ├─<statement>
72  | | | └─<assignment-statement>
73  | | |  ├─<variable>
74  | | |  | └─ident (flag)
75  | | |  ├─becomes
76  | | |  └─<expression>
77  | | |   └─<simple-expression>
78  | | |    ├─<term>
79  | | |    | └─<factor>
80  | | |    |  ├─notsy
81  | | |    |  └─<factor>
82  | | |    |    └─ident (flag)
83  | | |    ├─<additive-operator>
84  | | |    | └─orsy
85  | | |    └─<term>
86  | | |     ├─<factor>
87  | | |     | ├─lparent
88  | | |     | ├─<expression>
89  | | |     | | ├─<simple-expression>
90  | | |     | | | └─<term>
91  | | |     | | |  └─<factor>
92  | | |     | | |    └─ident (a)
93  | | |     | | ├─<relational-operator>
94  | | |     | | | └─leq
95  | | |     | | └─<simple-expression>
96  | | |     | |  └─<term>
97  | | |     | |   └─<factor>
98  | | |     | |     └─ident (b)
99  | | |     | └─rparent
100 | | |     ├─<multiplicative-operator>
101 | | |     | └─andsy
102 | | |     └─<factor>
103 | | |      ├─notsy
104 | | |      └─<factor>
105 | | |       ├─lparent
106 | | |       ├─<expression>
107 | | |       | ├─<simple-expression>
108 | | |       | | └─<term>
109 | | |       | |  └─<factor>
110 | | |       | |    └─ident (c)
111 | | |       | ├─<relational-operator>
112 | | |       | | └─eql
113 | | |       | └─<simple-expression>
114 | | |       |  └─<term>
115 | | |       |   └─<factor>
```

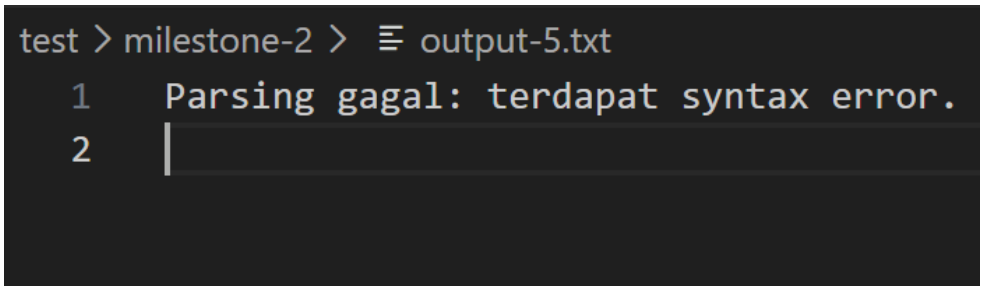
test > milestone-2 > output-4.txt

```
115   | | |       |   └─<factor>
116   | | |       |    └─realcon (0.0)
117   | | |       └─rparent
118   | | ├─semicolon
119   | | └─<statement>
120   | |  └─<repeat-statement>
121   | |    ├─repeatsy
122   | |    ├─<statement-list>
123   | |    | └─<statement>
124   | |    |  └─<assignment-statement>
125   | |    |   ├─<variable>
126   | |    |   | └─ident (a)
127   | |    |   ├─becomes
128   | |    |   └─<expression>
129   | |    |    └─<simple-expression>
130   | |    |     ├─<term>
131   | |    |     | └─<factor>
132   | |    |     |  └─ident (a)
133   | |    |     ├─<additive-operator>
134   | |    |     | └─minus
135   | |    |     └─<term>
136   | |    |      └─<factor>
137   | |    |       └─realcon (0.1)
138   | |    ├─untilsy
139   | |    └─<expression>
140   | |     ├─<simple-expression>
141   | |     | └─<term>
142   | |     |  └─<factor>
143   | |     |   └─ident (a)
144   | |     ├─<relational-operator>
145   | |     | └─lss
146   | |     └─<simple-expression>
147   | |      └─<term>
148   | |       └─<factor>
149   | |        └─realcon (0.0)
150   | └─endsy
151   └─period
```

5	**Input**

```
program UjiError;
var
  x: integer;
begin
  x := 5
```

	y := 10;
	
	if x > 0 then
	writeln('Halo'
	end.

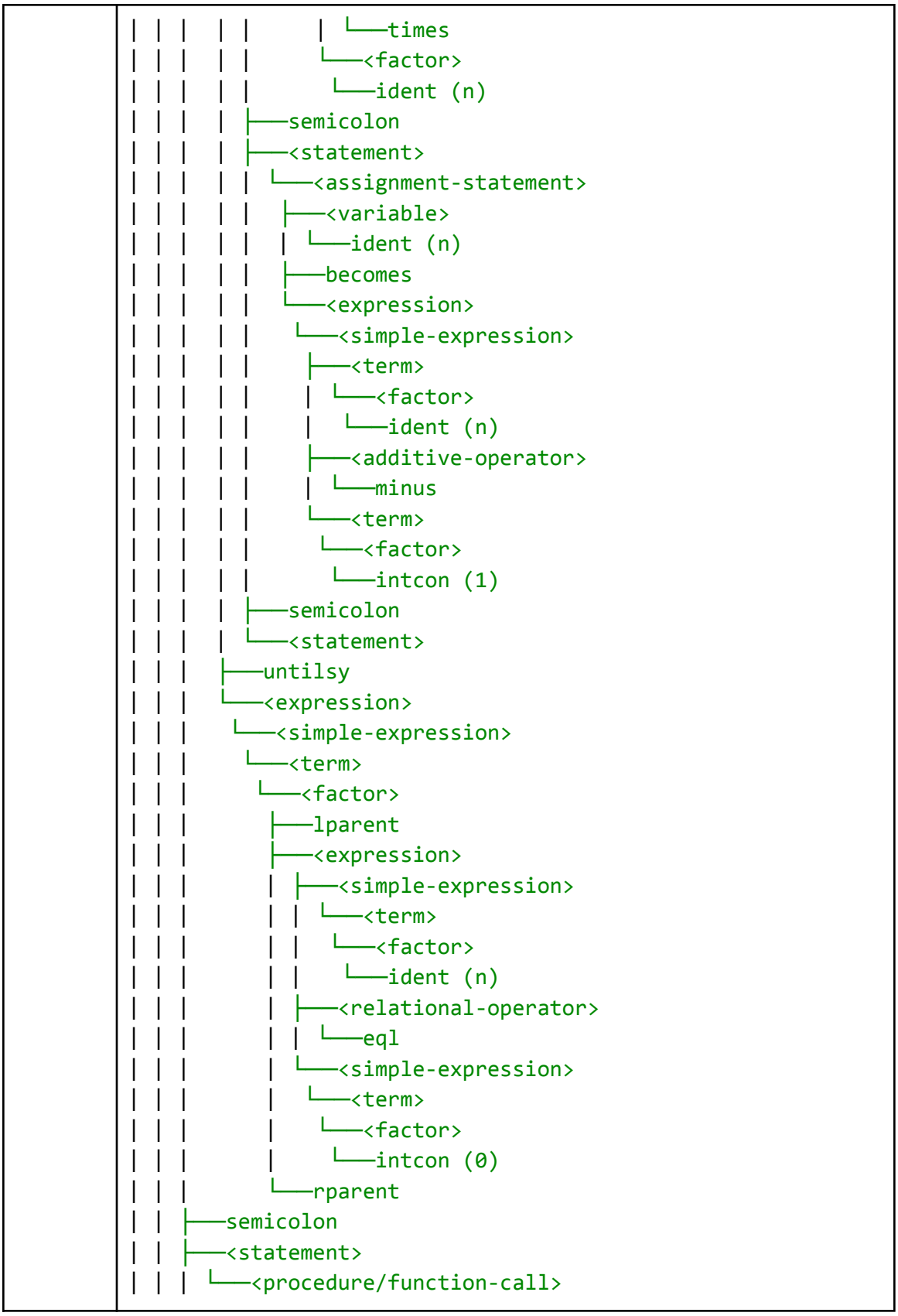
	Output
	Parsing gagal: terdapat syntax error.
	Screenshot Output
	
6	**Input**
	programsy ident (HitungFaktorial) semicolon varsy ident (n) comma ident (hasil) colon ident (integer) semicolon beginsy ident (n) becomes intcon (5) semicolon ident (hasil) becomes intcon (1) semicolon repeatsy ident (hasil) becomes

```
ident (hasil)
times
ident (n)
semicolon
ident (n)
becomes
ident (n)
minus
intcon (1)
semicolon
untilsy
lparent
ident (n)
eql
intcon (0)
rparent
semicolon
ident (writeln)
lparent
string ('Hasil = ')
comma
ident (hasil)
rparent
semicolon
endsy
period
```

Output

```
<program>
├──<program-header>
│  ├──programsy
│  ├──ident (HitungFaktorial)
│  └──semicolon
├──<declaration-part>
│  └──<var-declaration>
│     ├──varsy
│     ├──<identifier-list>
│     │  ├──ident (n)
│     │  ├──comma
│     │  └──ident (hasil)
│     ├──colon
│     ├──<type>
```

```
|  | └──ident (integer)
|  └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (n)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (5)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (hasil)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (1)
| | ├──semicolon
| | ├──<statement>
| | | └──<repeat-statement>
| | |  ├──repeatsy
| | |  ├──<statement-list>
| | |  | ├──<statement>
| | |  | | └──<assignment-statement>
| | |  | |  ├──<variable>
| | |  | |  | └──ident (hasil)
| | |  | |  ├──becomes
| | |  | |  └──<expression>
| | |  | |   └──<simple-expression>
| | |  | |    └──<term>
| | |  | |     ├──<factor>
| | |  | |     | └──ident (hasil)
| | |  | |     ├──<multiplicative-operator>
```


| | | | | | | | └──times
| | | | | | | └──<factor>
| | | | | | | └──ident (n)
| | | | | ├──semicolon
| | | | | ├──<statement>
| | | | | | └──<assignment-statement>
| | | | | | ├──<variable>
| | | | | | | └──ident (n)
| | | | | | ├──becomes
| | | | | | └──<expression>
| | | | | | └──<simple-expression>
| | | | | | ├──<term>
| | | | | | | └──<factor>
| | | | | | | └──ident (n)
| | | | | | ├──<additive-operator>
| | | | | | | └──minus
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──intcon (1)
| | | | | ├──semicolon
| | | | | | └──<statement>
| | | | ├──untilsy
| | | | └──<expression>
| | | | └──<simple-expression>
| | | | └──<term>
| | | | └──<factor>
| | | | ├──lparent
| | | | ├──<expression>
| | | | | ├──<simple-expression>
| | | | | | └──<term>
| | | | | | └──<factor>
| | | | | | └──ident (n)
| | | | | ├──<relational-operator>
| | | | | | └──eql
| | | | | └──<simple-expression>
| | | | | └──<term>
| | | | | └──<factor>
| | | | | └──intcon (0)
| | | | └──rparent
| | ├──semicolon
| | ├──<statement>
| | | └──<procedure/function-call>

```
| | |   ├──ident (writeln)
| | |   ├──lparent
| | |   ├──<parameter-list>
| | |   | ├──<expression>
| | |   | | └──<simple-expression>
| | |   | |   └──<term>
| | |   | |    └──<factor>
| | |   | |     └──string ('Hasil = ')
| | |   | ├──comma
| | |   | └──<expression>
| | |   |   └──<simple-expression>
| | |   |    └──<term>
| | |   |     └──<factor>
| | |   |      └──ident (hasil)
| | |   └──rparent
| | ├──semicolon
| | └──<statement>
| └──endsy
└──period
```

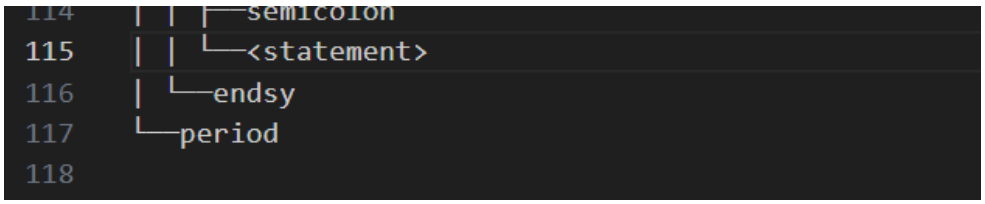
Screenshot Output

test > milestone-2 > output-6.txt

```
<program>
├──<program-header>
| ├──programsy
| ├──ident (HitungFaktorial)
| └──semicolon
├──<declaration-part>
| └──<var-declaration>
|  ├──varsy
|  ├──<identifier-list>
|  | ├──ident (n)
|  | ├──comma
|  | └──ident (hasil)
|  ├──colon
|  ├──<type>
|  | └──ident (integer)
|  └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (n)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (5)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (hasil)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (1)
| | ├──semicolon
| | ├──<statement>
| | | └──<repeat-statement>
| | |  ├──repeatsy
| | |  ├──<statement-list>
| | |  | ├──<statement>
| | |  | | └──<assignment-statement>
| | |  | |  ├──<variable>
| | |  | |  | └──ident (hasil)
| | |  | |  ├──becomes
| | |  | |  └──<expression>
| | |  | |   └──<simple-expression>
| | |  | |    └──<term>
| | |  | |     ├──<factor>
| | |  | |     | └──ident (hasil)
| | |  | |     ├──<multiplicative-operator>
| | |  | |     | └──times
| | |  | |     └──<factor>
```

test > milestone-2 > output-6.txt

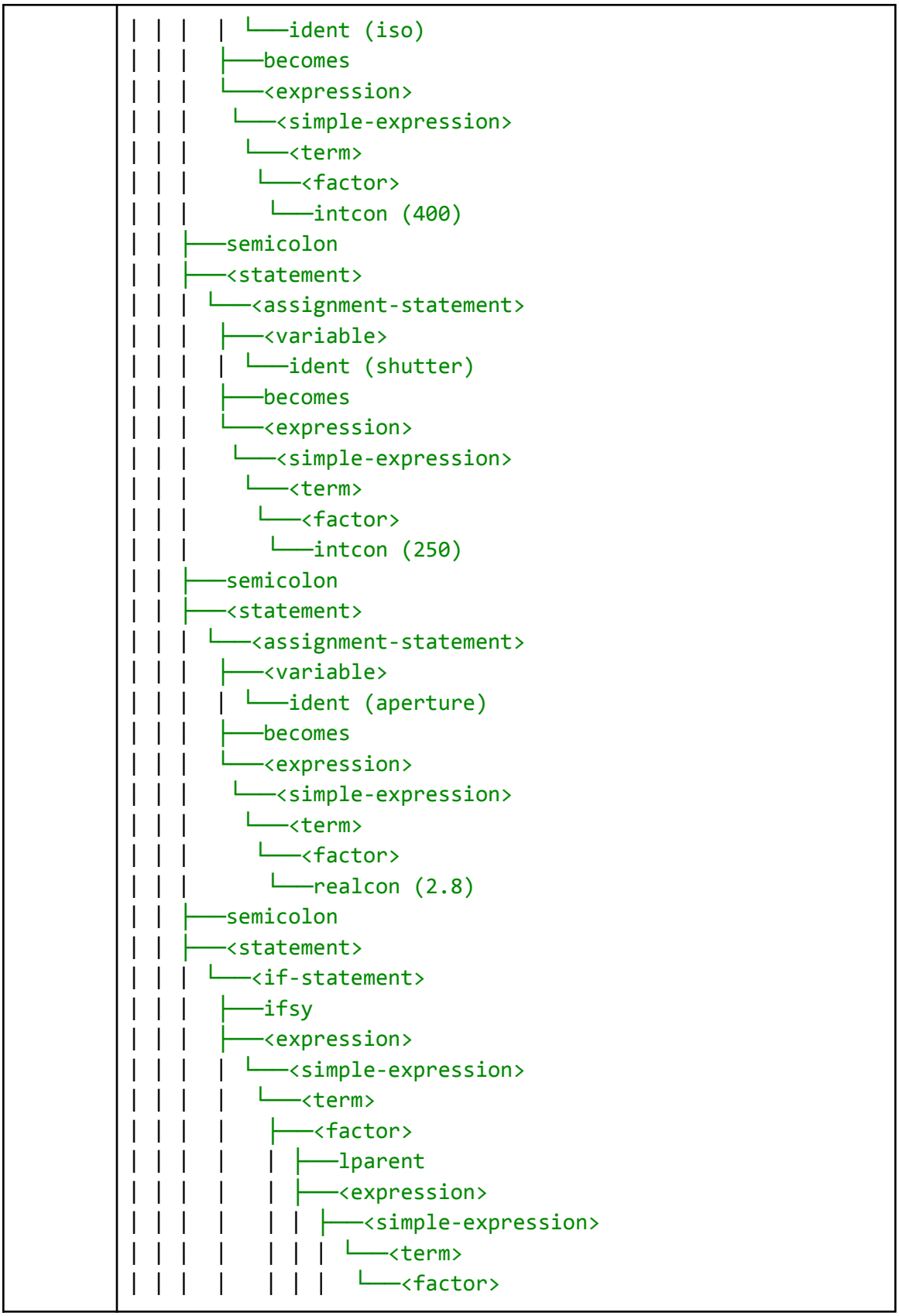
```
58  | | | | | |     └─<factor>
59  | | | | | |       └─ident (n)
60  | | | | | ├─semicolon
61  | | | | | ├─<statement>
62  | | | | | | └─<assignment-statement>
63  | | | | | |   ├─<variable>
64  | | | | | |   | └─ident (n)
65  | | | | | |   ├─becomes
66  | | | | | |   └─<expression>
67  | | | | | |     └─<simple-expression>
68  | | | | | |       ├─<term>
69  | | | | | |       | └─<factor>
70  | | | | | |       |   └─ident (n)
71  | | | | | |       ├─<additive-operator>
72  | | | | | |       | └─minus
73  | | | | | |       └─<term>
74  | | | | | |         └─<factor>
75  | | | | | |           └─intcon (1)
76  | | | | | ├─semicolon
77  | | | | | └─<statement>
78  | | | | ├─untilsy
79  | | | | └─<expression>
80  | | | |   └─<simple-expression>
81  | | | |     └─<term>
82  | | | |       └─<factor>
83  | | | |         ├─lparent
84  | | | |         ├─<expression>
85  | | | |         | ├─<simple-expression>
86  | | | |         | | └─<term>
87  | | | |         | |   └─<factor>
88  | | | |         | |     └─ident (n)
89  | | | |         | ├─<relational-operator>
90  | | | |         | | └─eql
91  | | | |         | └─<simple-expression>
92  | | | |         |   └─<term>
93  | | | |         |     └─<factor>
94  | | | |         |       └─intcon (0)
95  | | | |         └─rparent
96  | | ├─semicolon
97  | | ├─<statement>
98  | | | └─<procedure/function-call>
99  | | |   ├─ident (writeln)
100 | | |   ├─lparent
101 | | |   ├─<parameter-list>
102 | | |   | ├─<expression>
103 | | |   | | └─<simple-expression>
104 | | |   | |   └─<term>
105 | | |   | |     └─<factor>
106 | | |   | |       └─string ('Hasil = ')
107 | | |   | ├─comma
108 | | |   | └─<expression>
109 | | |   |   └─<simple-expression>
110 | | |   |     └─<term>
111 | | |   |       └─<factor>
112 | | |   |         └─ident (hasil)
113 | | |   └─rparent
114 | | ├─semicolon
115 | | └─<statement>
```

	
7	**Input**
	programsy ident (HitungExposure) semicolon varsy ident (iso) comma ident (shutter) colon ident (integer) semicolon ident (aperture) colon ident (real) semicolon beginsy ident (iso) becomes intcon (400) semicolon ident (shutter) becomes intcon (250) semicolon ident (aperture) becomes realcon (2.8) semicolon ifsy lparent ident (iso) geq intcon (800) rparent andsy lparent

```
ident (aperture)
leq
realcon (1.4)
rparent
thensy
ident (writeln)
lparent
string ('Foto overexposed')
rparent
semicolon
endsy
period
```

Output

```
<program>
├──<program-header>
│  ├──programsy
│  ├──ident (HitungExposure)
│  └──semicolon
├──<declaration-part>
│  └──<var-declaration>
│     ├──varsy
│     ├──<identifier-list>
│     │  ├──ident (iso)
│     │  ├──comma
│     │  └──ident (shutter)
│     ├──colon
│     ├──<type>
│     │  └──ident (integer)
│     ├──semicolon
│     ├──<identifier-list>
│     │  └──ident (aperture)
│     ├──colon
│     ├──<type>
│     │  └──ident (real)
│     └──semicolon
├──<compound-statement>
│  ├──beginsy
│  ├──<statement-list>
│  │  ├──<statement>
│  │  │  └──<assignment-statement>
│  │  │     ├──<variable>
```


| | | | └──ident (iso)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | └──<term>
| | | └──<factor>
| | | └──intcon (400)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (shutter)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | └──<term>
| | | └──<factor>
| | | └──intcon (250)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (aperture)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | └──<term>
| | | └──<factor>
| | | └──realcon (2.8)
| | ├──semicolon
| | ├──<statement>
| | | └──<if-statement>
| | | ├──ifsy
| | | ├──<expression>
| | | | └──<simple-expression>
| | | | └──<term>
| | | | ├──<factor>
| | | | | ├──lparent
| | | | | ├──<expression>
| | | | | | ├──<simple-expression>
| | | | | | | └──<term>
| | | | | | | └──<factor>

```
| | |   |    | | |    └──ident (iso)
| | |   |    | | ├──<relational-operator>
| | |   |    | | | └──geq
| | |   |    | | └──<simple-expression>
| | |   |    | |  └──<term>
| | |   |    | |   └──<factor>
| | |   |    | |    └──intcon (800)
| | |   |    | └──rparent
| | |   |    ├──<multiplicative-operator>
| | |   |    | └──andsy
| | |   |    └──<factor>
| | |   |     ├──lparent
| | |   |     ├──<expression>
| | |   |     | ├──<simple-expression>
| | |   |     | | └──<term>
| | |   |     | |  └──<factor>
| | |   |     | |   └──ident (aperture)
| | |   |     | ├──<relational-operator>
| | |   |     | | └──leq
| | |   |     | └──<simple-expression>
| | |   |     |  └──<term>
| | |   |     |   └──<factor>
| | |   |     |    └──realcon (1.4)
| | |   |     └──rparent
| | |   ├──thensy
| | |   └──<statement>
| | |    └──<procedure/function-call>
| | |     ├──ident (writeln)
| | |     ├──lparent
| | |     ├──<parameter-list>
| | |     | └──<expression>
| | |     |  └──<simple-expression>
| | |     |   └──<term>
| | |     |    └──<factor>
| | |     |     └──string ('Foto overexposed')
| | |     └──rparent
| | ├──semicolon
| | └──<statement>
| └──endsy
└──period
```

Screenshot Output

test > milestone-2 > output-7.txt

```
<program>
├──<program-header>
| ├──programsy
| ├──ident (HitungExposure)
| └──semicolon
├──<declaration-part>
| └──<var-declaration>
|  ├──varsy
|  ├──<identifier-list>
|  | ├──ident (iso)
|  | ├──comma
|  | └──ident (shutter)
|  ├──colon
|  ├──<type>
|  | └──ident (integer)
|  ├──semicolon
|  ├──<identifier-list>
|  | └──ident (aperture)
|  ├──colon
|  ├──<type>
|  | └──ident (real)
|  └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (iso)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (400)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (shutter)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (250)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | |  ├──<variable>
| | |  | └──ident (aperture)
| | |  ├──becomes
| | |  └──<expression>
| | |   └──<simple-expression>
| | |    └──<term>
| | |     └──<factor>
| | |      └──realcon (2.8)
| | ├──semicolon
```

test > milestone-2 > output-7.txt

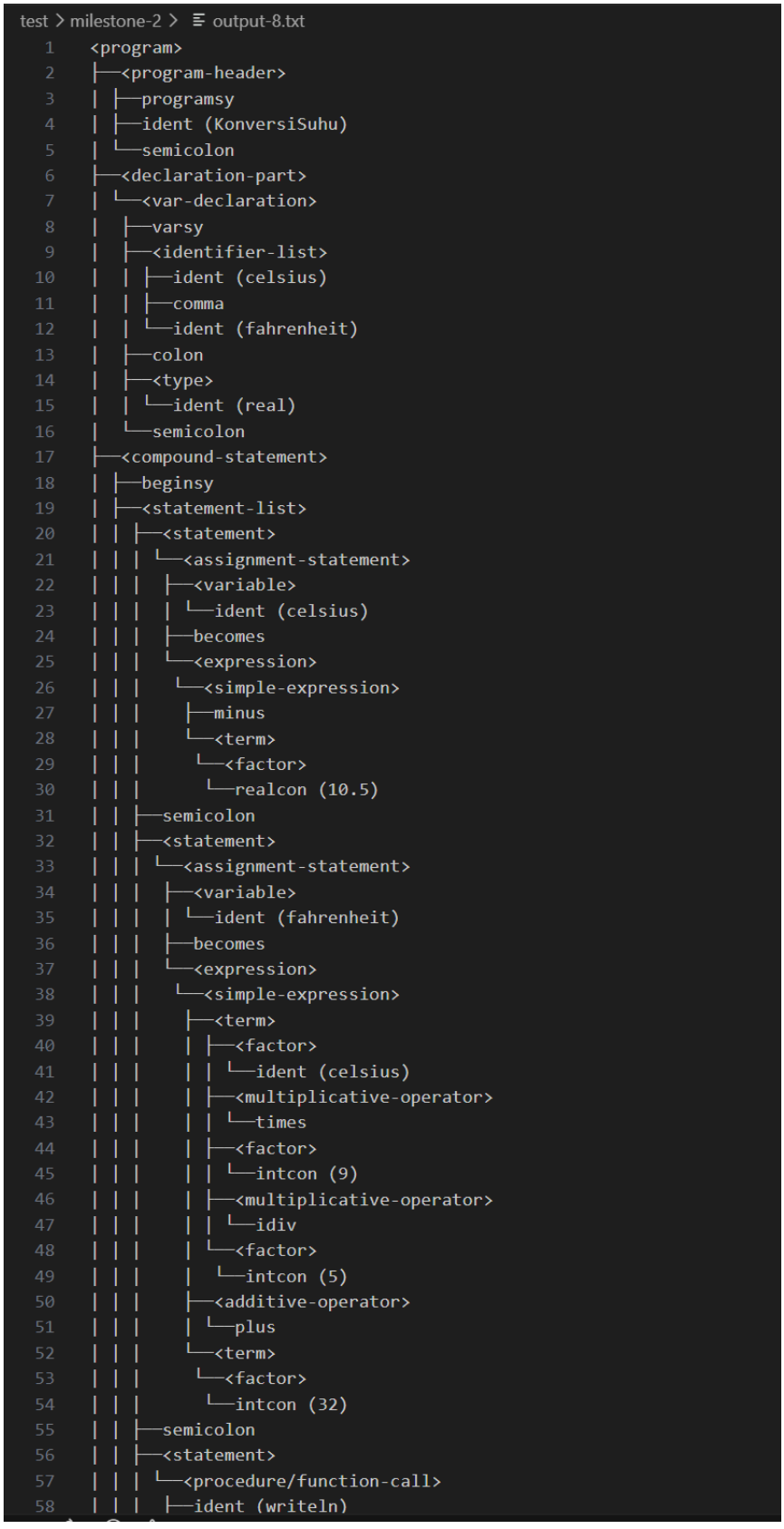
```
58  | | ├──semicolon
59  | | ├──<statement>
60  | | | └──<if-statement>
61  | | |  ├──ifsy
62  | | |  ├──<expression>
63  | | |  | └──<simple-expression>
64  | | |  |  └──<term>
65  | | |  |   ├──<factor>
66  | | |  |   | ├──lparent
67  | | |  |   | ├──<expression>
68  | | |  |   | | ├──<simple-expression>
69  | | |  |   | | | └──<term>
70  | | |  |   | | |  └──<factor>
71  | | |  |   | | |   └──ident (iso)
72  | | |  |   | | ├──<relational-operator>
73  | | |  |   | | | └──geq
74  | | |  |   | | └──<simple-expression>
75  | | |  |   | |  └──<term>
76  | | |  |   | |   └──<factor>
77  | | |  |   | |    └──intcon (800)
78  | | |  |   | └──rparent
79  | | |  |   ├──<multiplicative-operator>
80  | | |  |   | └──andsy
81  | | |  |   └──<factor>
82  | | |  |    ├──lparent
83  | | |  |    ├──<expression>
84  | | |  |    | ├──<simple-expression>
85  | | |  |    | | └──<term>
86  | | |  |    | |  └──<factor>
87  | | |  |    | |   └──ident (aperture)
88  | | |  |    | ├──<relational-operator>
89  | | |  |    | | └──leq
90  | | |  |    | └──<simple-expression>
91  | | |  |    |  └──<term>
92  | | |  |    |   └──<factor>
93  | | |  |    |    └──realcon (1.4)
94  | | |  |    └──rparent
95  | | |  ├──thensy
96  | | |  └──<statement>
97  | | |   └──<procedure/function-call>
98  | | |    ├──ident (writeln)
99  | | |    ├──lparent
100 | | |    ├──<parameter-list>
101 | | |    | └──<expression>
102 | | |    |  └──<simple-expression>
103 | | |    |   └──<term>
104 | | |    |    └──<factor>
105 | | |    |     └──string ('Foto overexposed')
106 | | |    └──rparent
107 | | ├──semicolon
108 | | └──<statement>
109 | └──endsy
110 └──period
```

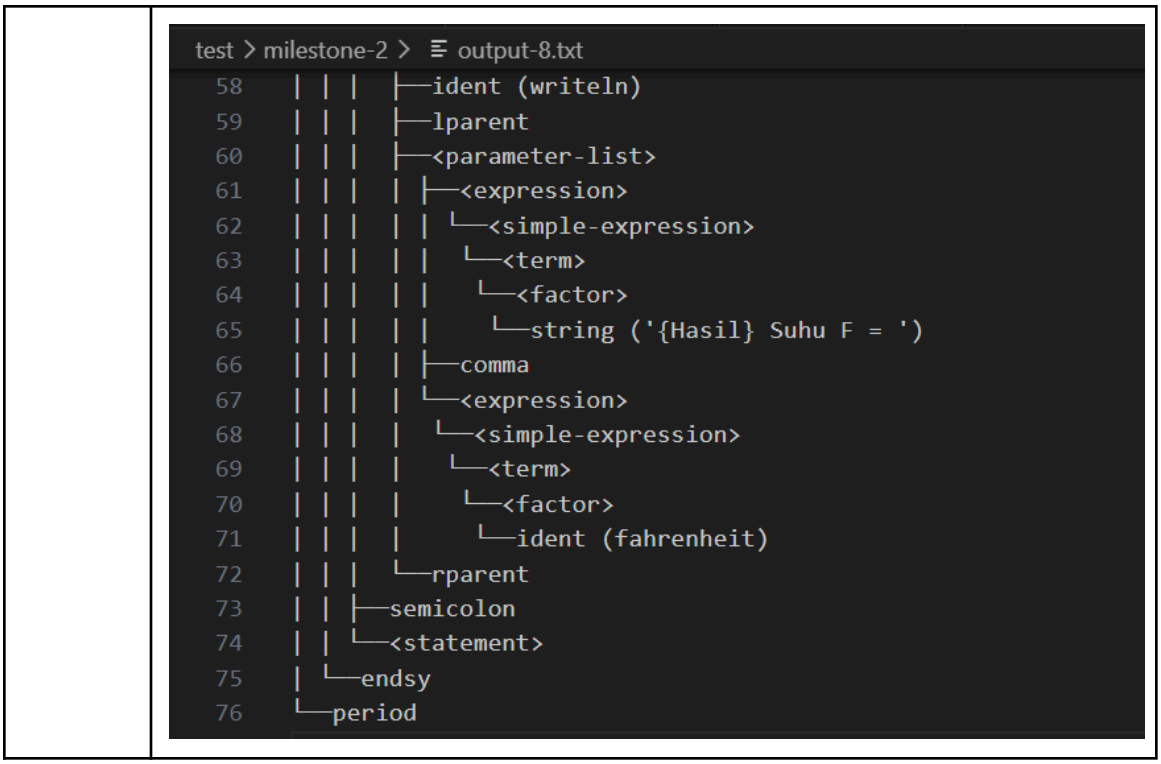
8	Input
	programsy ident (KonversiSuhu) semicolon varsy ident (celsius) comma ident (fahrenheit) colon ident (real) semicolon beginsy ident (celsius) becomes minus realcon (10.5) semicolon ident (fahrenheit) becomes ident (celsius) times intcon (9) idiv intcon (5) plus intcon (32) semicolon ident (writeln) lparent string ('{Hasil} Suhu F = ') comma ident (fahrenheit) rparent semicolon endsy period
	Output
	<program> ├──<program-header> │ ├──programsy

```
| ├──ident (KonversiSuhu)
| └──semicolon
├──<declaration-part>
| └──<var-declaration>
|   ├──varsy
|   ├──<identifier-list>
|   | ├──ident (celsius)
|   | ├──comma
|   | └──ident (fahrenheit)
|   ├──colon
|   ├──<type>
|   | └──ident (real)
|   └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | |   ├──<variable>
| | |   | └──ident (celsius)
| | |   ├──becomes
| | |   └──<expression>
| | |    └──<simple-expression>
| | |     ├──minus
| | |     └──<term>
| | |      └──<factor>
| | |       └──realcon (10.5)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | |   ├──<variable>
| | |   | └──ident (fahrenheit)
| | |   ├──becomes
| | |   └──<expression>
| | |    └──<simple-expression>
| | |     ├──<term>
| | |     | ├──<factor>
| | |     | | └──ident (celsius)
| | |     | ├──<multiplicative-operator>
| | |     | | └──times
| | |     | ├──<factor>
| | |     | | └──intcon (9)
```

```
| | |    | ├──<multiplicative-operator>
| | |    | | └──idiv
| | |    | └──<factor>
| | |    |  └──intcon (5)
| | |    ├──<additive-operator>
| | |    | └──plus
| | |    └──<term>
| | |     └──<factor>
| | |      └──intcon (32)
| | ├──semicolon
| | ├──<statement>
| | | └──<procedure/function-call>
| | |  ├──ident (writeln)
| | |  ├──lparent
| | |  ├──<parameter-list>
| | |  | ├──<expression>
| | |  | | └──<simple-expression>
| | |  | |  └──<term>
| | |  | |   └──<factor>
| | |  | |    └──string ('{Hasil} Suhu F = ')
| | |  | ├──comma
| | |  | └──<expression>
| | |  |  └──<simple-expression>
| | |  |   └──<term>
| | |  |    └──<factor>
| | |  |     └──ident (fahrenheit)
| | |  └──rparent
| | ├──semicolon
| | └──<statement>
| └──endsy
└──period
```

Screenshot Output


test > milestone-2 > output-8.txt
<program>
├──<program-header>
| ├──programsy
| ├──ident (KonversiSuhu)
| └──semicolon
├──<declaration-part>
| └──<var-declaration>
| ├──varsy
| ├──<identifier-list>
| | ├──ident (celsius)
| | ├──comma
| | └──ident (fahrenheit)
| ├──colon
| ├──<type>
| | └──ident (real)
| └──semicolon
├──<compound-statement>
| ├──beginsy
| ├──<statement-list>
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (celsius)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | ├──minus
| | | └──<term>
| | | └──<factor>
| | | └──realcon (10.5)
| | ├──semicolon
| | ├──<statement>
| | | └──<assignment-statement>
| | | ├──<variable>
| | | | └──ident (fahrenheit)
| | | ├──becomes
| | | └──<expression>
| | | └──<simple-expression>
| | | ├──<term>
| | | | ├──<factor>
| | | | | └──ident (celsius)
| | | | ├──<multiplicative-operator>
| | | | | └──times
| | | | ├──<factor>
| | | | | └──intcon (9)
| | | | ├──<multiplicative-operator>
| | | | | └──idiv
| | | | └──<factor>
| | | | └──intcon (5)
| | | ├──<additive-operator>
| | | | └──plus
| | | └──<term>
| | | └──<factor>
| | | └──intcon (32)
| | ├──semicolon
| | ├──<statement>
| | | └──<procedure/function-call>
| | | ├──ident (writeln)



E. KESIMPULAN DAN SARAN

1. Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa tahapan *Syntax Analysis* (Milestone 2) untuk *Arion Compiler* berhasil dibangun dan diimplementasikan dengan baik. Program kompilator sukses memanfaatkan algoritma *Recursive Descent* untuk mengenali struktur gramatikal dari kumpulan token dan secara akurat mengonstruksinya ke dalam representasi hierarkis *Parse Tree* yang memuat seluruh simbol terminal dan non-terminal. Modul juga terbukti tangguh (*robust*) dalam membedakan format masukan, sehingga dapat menerima baik fail *source code* Arion maupun fail berformat daftar token hasil proses analisis leksikal pada Milestone 1. Fungsi *error handling* pada tingkatan sintaks juga telah berjalan sebagaimana semestinya.

2. Saran

Meskipun *parser* telah berjalan dengan fungsionalitas yang disyaratkan, terdapat beberapa area yang dapat dikembangkan untuk perbaikan di *milestone* berikutnya:

a. Hasil dari proses ini adalah *Parse Tree* yang masih mengandung banyak informasi *verbose* (seperti terminal operator titik koma, tanda kurung, dll.).

Disarankan untuk segera mentransformasikan hasil ini menjadi *Abstract Syntax Tree* (AST) untuk menyederhanakan pohon kedepannya, sehingga lebih efisien saat digunakan di fase analisis semantik.

b. Pesan ralat (*error message*) saat ini sudah informatif terkait jenis token, namun alangkah lebih baik jika pesan tersebut dilengkapi dengan penunjuk lokasi baris (line) dan kolom (column) spesifik di mana *syntax error* tersebut terjadi agar pengguna lebih mudah memperbaiki kode sumbernya. (Dibutuhkan integrasi informasi baris/kolom dari lexer).
c. Mekanisme parser saat ini berhenti (*halt*) pada saat satu kesalahan sintaksis pertama kali ditemukan. Disarankan untuk menambahkan metode sinkronisasi (*error recovery* seperti *panic mode*) sehingga kompilator dapat terus menganalisis dan mendeteksi berbagai ralat sintaksis sekaligus di seluruh program dalam sekali jalan.

F. LAMPIRAN

Github: https://github.com/Rmadhian/DGN-Tubes-IF2224-2026

Tabel Kontribusi:

NIM	Kontribusi	Persentase
13523150	Mengimplementasi parser_stmt.cpp & laporan	33,33%
13524117	-	00,00%
13524120	Mengimplementasi parser_decl.cpp, membantu integrasi program & laporan	33,33%
13524126	Mengimplementasi parser_expr.cpp, integrasi program & laporan	33,34%

G. REFERENSI

Aiken, A. (2012). *Lexical analysis* [PDF slides]. Stanford University CS143 Archive. https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/02/Slides02.pdf

GeeksforGeeks. (2023, 14 Juni). *Recursive descent parser*. https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/